

Machines en berekenbaarheid

Introduction to Theoretical Computer Science

Els Laenens

Thanks to G. Grahne for an earlier version of these slides.

Context-Free Grammars and Languages

- We have seen that many languages cannot be regular (e.g. $\{a^n b^n : n \geq 1\}$). Thus we need to consider larger classes of languages.
- *Context-Free Languages* (CFL's) played a central role in natural languages since the 1950's, and in compilers since the 1960's.
- *Context-Free Grammars* (CFG's) are the basis of BNF (Backus-Naur/Normal Form), widely used for specifying the syntax of programming languages.
- Today CFL's are increasingly important for XML and their DTD's.

We'll look at: CFG's, the languages they generate, parse trees, pushdown automata, and closure properties of CFL's.

Informal example of CFG's

Consider $L_{pal} = \{w \in \Sigma^* : w = w^R\}$

For example $otto \in L_{pal}$, $madamimadam \in L_{pal}$.

In Finnish language e.g. $saippuakauppias \in L_{pal}$
("soap-merchant")

L_{pal} is not regular. Pumping lemma with constant n and string $0^n 1 0^n$.

Let $\Sigma = \{0, 1\}$.

Inductive definition of L_{pal} :

Basis: $\epsilon, 0$ and 1 are palindromes.

Induction: If w is a palindrome, so are $0w0$ and $1w1$.

Circumscription: Nothing else is a palindrome.

A *context-free grammar* (CFG) is a formal mechanism for definitions such as the one for L_{pal} .

1. $P \rightarrow \epsilon$
2. $P \rightarrow 0$
3. $P \rightarrow 1$
4. $P \rightarrow 0P0$
5. $P \rightarrow 1P1$

1–5 are *productions* or *rules*

0 and 1 are *terminals*

P is a *variable* or *nonterminal* (or *syntactic category*)

P is in this grammar also the *start symbol*

Formal definition of CFG's

A *context-free grammar* is a quadruple

$$G = (V, T, P, S)$$

where V is a finite set of *variables* or *nonterminals*.

T is a finite set of *terminals*.

P is a finite set of *productions* or *rules* of the form $A \rightarrow \alpha$, where A is a variable and $\alpha \in (V \cup T)^*$

S is a designated variable called the *start symbol*.

Example: $G_{pal} = (\{P\}, \{0, 1\}, A, P)$, where $A = \{P \rightarrow \epsilon, P \rightarrow 0, P \rightarrow 1, P \rightarrow 0P0, P \rightarrow 1P1\}$.

Sometimes we group productions with the same head, e.g. $A = \{P \rightarrow \epsilon | 0 | 1 | 0P0 | 1P1\}$.

Example: (simple) expressions in a typical programming language.

Operators: $+$ and $*$. Arguments: identifiers, i.e. strings in $L((a + b)(a + b + 0 + 1)^*)$.

The expressions are defined by the expressions grammar

$$G = (\{E, I\}, T, P, E)$$

where $T = \{+, *, (,), a, b, 0, 1\}$ and P is the following set of productions:

1. $E \rightarrow I$
2. $E \rightarrow E + E$
3. $E \rightarrow E * E$
4. $E \rightarrow (E)$
5. $I \rightarrow a$
6. $I \rightarrow b$
7. $I \rightarrow Ia$
8. $I \rightarrow Ib$
9. $I \rightarrow I0$
10. $I \rightarrow I1$

Derivations using grammars

- *Recursive inference*, using productions from body to head

Definition of recursive inference:

Basis: Let $A \rightarrow w \in P$ and $w \in T^*$. Then $w \in L(A)$.

Induction: Let $A \rightarrow X_1X_2...X_k$ and $\forall X_i \in V$, let $w_i \in L(X_i)$ and $\forall X_i \in T$, let $w_i = X_i$. Then $w_1w_2...w_k \in L(A)$.

Example

	String Inferred	in language of	Production used	String(s) used
(i)	a	I	5	-
(ii)	b	I	6	-
(iii)	$b0$	I	9	(ii)
(iv)	$b00$	I	9	(iii)
(v)	a	E	1	(i)
(vi)	$b00$	E	1	(iv)
(vii)	$a + b00$	E	2	(v), (vi)
(viii)	$(a + b00)$	E	4	(vii)
(ix)	$a * (a + b00)$	E	3	(v), (viii)

- *Derivations*, using productions from head to body.

Let $G = (V, T, P, S)$ be a CFG, $A \in V$, $\{\alpha, \beta\} \subset (V \cup T)^*$, and $A \rightarrow \gamma \in P$.

Then we write

$$\alpha A \beta \Rightarrow_G \alpha \gamma \beta$$

or, if G is understood

$$\alpha A \beta \Rightarrow \alpha \gamma \beta$$

and say that $\alpha A \beta$ *derives* $\alpha \gamma \beta$.

We define $\overset{*}{\Rightarrow}$ to be the reflexive and transitive closure of $\Rightarrow$:

Basis: Let $\alpha \in (V \cup T)^*$. Then $\alpha \overset{*}{\Rightarrow} \alpha$.

Induction: If $\alpha \overset{*}{\Rightarrow} \beta$, and $\beta \Rightarrow \gamma$, then $\alpha \overset{*}{\Rightarrow} \gamma$.

Example: Derivation of $a * (a + b00)$ from E in the expressions grammar:

$$\begin{aligned}
 E &\Rightarrow E * E \Rightarrow I * E \Rightarrow a * E \Rightarrow a * (E) \Rightarrow \\
 a * (E + E) &\Rightarrow a * (I + E) \Rightarrow a * (a + E) \Rightarrow a * (a + I) \Rightarrow \\
 a * (a + I0) &\Rightarrow a * (a + I00) \Rightarrow a * (a + b00)
 \end{aligned}$$

Note: At each step we might have several rules to choose from, e.g.

$$\begin{aligned}
 I * E &\Rightarrow a * E \Rightarrow a * (E), \text{ versus} \\
 I * E &\Rightarrow I * (E) \Rightarrow a * (E).
 \end{aligned}$$

Note2: Not all choices lead to successful derivations of a particular string, for instance

$$E \Rightarrow E + E$$

won't lead to a derivation of $a * (a + b00)$.

Leftmost and Rightmost Derivations

Leftmost derivation $\Rightarrow_{lm}$: Always replace the leftmost variable by one of its rule-bodies.

Rightmost derivation $\Rightarrow_{rm}$: Always replace the rightmost variable by one of its rule-bodies.

Leftmost: The derivation on the previous slide.

Rightmost:

$$E \Rightarrow_{rm} E * E \Rightarrow_{rm}$$

$$E * (E) \Rightarrow_{rm} E * (E + E) \Rightarrow_{rm} E * (E + I) \Rightarrow_{rm} E * (E + I0)$$

$$\Rightarrow_{rm} E * (E + I00) \Rightarrow_{rm} E * (E + b00) \Rightarrow_{rm} E * (I + b00)$$

$$\Rightarrow_{rm} E * (a + b00) \Rightarrow_{rm} I * (a + b00) \Rightarrow_{rm} a * (a + b00)$$

We can conclude that $E \xRightarrow{*}_{rm} a * (a + b00)$

The Language of a Grammar

If $G = (V, T, P, S)$ is a CFG, then the *language of G* is

$$L(G) = \{w \in T^* : S \xRightarrow[G]{*} w\}$$

i.e. the set of strings over T^* derivable from the start symbol.

If G is a CFG, we call $L(G)$ a *context-free language (CFL)*.

Example: $L(G_{pal})$ is a context-free language.

Theorem 5.7:

$$L(G_{pal}) = \{w \in \{0, 1\}^* : w = w^R\}$$

Proof: ($\supseteq$ -direction.) Suppose $w = w^R$. We show by induction on $|w|$ that $w \in L(G_{pal})$, i.e. $P \xRightarrow[G]{*} w$.

Basis: $|w| = 0$, or $|w| = 1$. Then w is $\epsilon, 0$, or 1 . Since $P \rightarrow \epsilon, P \rightarrow 0$, and $P \rightarrow 1$ are productions, we conclude that $P \xRightarrow[G]{*} w$ in all base cases.

Induction: Suppose $|w| \geq 2$. Since $w = w^R$, we have $w = 0x0$, or $w = 1x1$, and $x = x^R$.

If $w = 0x0$ we know from the IH that $P \xRightarrow{*} x$. Then

$$P \Rightarrow 0P0 \xRightarrow{*} 0x0 = w$$

Thus $w \in L(G_{pal})$.

The case for $w = 1x1$ is similar.

($\subseteq$ -direction.) We assume that $w \in L(G_{pal})$ and must show that $w = w^R$.

Since $w \in L(G_{pal})$, we have $P \xRightarrow{*} w$.

We do an induction of the length of $\xRightarrow{*}$.

Basis: The derivation $P \xRightarrow{*} w$ is done in one step.

Then w must be ϵ , 0 , or 1 , all palindromes.

Induction: Let $n \geq 1$, and suppose the derivation takes $n + 1$ steps. Then we must have

$$P \Rightarrow 0P0 \xRightarrow{*} 0x0 = w$$

or

$$P \Rightarrow 1P1 \xRightarrow{*} 1x1 = w$$

where the second derivation is done in n steps.

By the IH x is a palindrome, and the inductive proof is complete.

Sentential Forms

Let $G = (V, T, P, S)$ be a CFG, and $\alpha \in (V \cup T)^*$.

If

$$S \xRightarrow{*} \alpha$$

we say that α is a *sentential form*.

If $S \xRightarrow[lm]{*} \alpha$ we say that α is a *left-sentential form*,

and if $S \xRightarrow[rm]{*} \alpha$ we say that α is a *right-sentential form*

Note: $L(G)$ is those sentential forms that are in T^* .

Example: Take the expressions grammar G .
Then $E * (I + E)$ is a sentential form since

$$E \Rightarrow E * E \Rightarrow E * (E) \Rightarrow E * (E + E) \Rightarrow E * (I + E)$$

This derivation is neither leftmost, nor rightmost

Example: $a * E$ is a left-sentential form, since

$$E \underset{lm}{\Rightarrow} E * E \underset{lm}{\Rightarrow} I * E \underset{lm}{\Rightarrow} a * E$$

Example: $E * (E + E)$ is a right-sentential form, since

$$E \underset{rm}{\Rightarrow} E * E \underset{rm}{\Rightarrow} E * (E) \underset{rm}{\Rightarrow} E * (E + E)$$

Parse Trees

- If $w \in L(G)$, for some CFG, then w has a *parse tree*, which tells us the (syntactic) structure of w
- w could be a program, a SQL-query, an XML-document, etc.
- Parse trees are an alternative representation to derivations and recursive inferences.
- There can be several parse trees for the same string
- Ideally there should be only one parse tree (the “true” structure) for each string, i.e. the language should be *unambiguous*.
- Unfortunately, we cannot always remove the ambiguity.

Constructing Parse Trees

Let $G = (V, T, P, S)$ be a CFG. A tree is a *parse tree* for G if:

1. Each interior node is labelled by a variable in V .
2. Each leaf is labelled by a symbol in $V \cup T \cup \{\epsilon\}$. Any ϵ -labelled leaf is the only child of its parent.
3. If an interior node is labelled A , and its children (from left to right) labelled

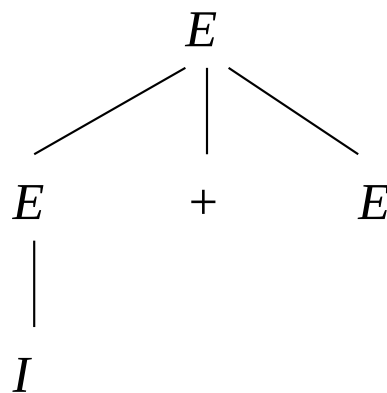
$$X_1, X_2, \dots, X_k,$$

then $A \rightarrow X_1 X_2 \dots X_k \in P$.

Example: In the grammar

1. $E \rightarrow I$
2. $E \rightarrow E + E$
3. $E \rightarrow E * E$
4. $E \rightarrow (E)$
- $\vdots$

the following is a parse tree:

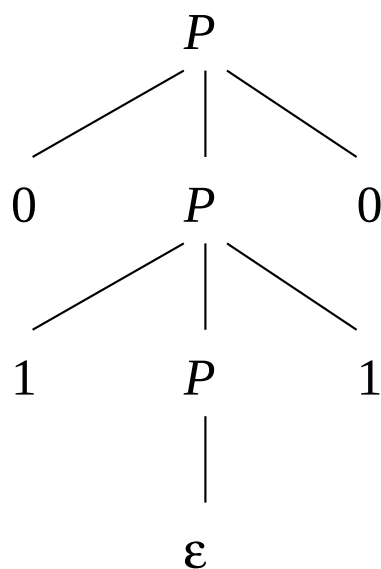


This parse tree shows the derivation $E \xRightarrow{*} I + E$

Example: In the grammar

1. $P \rightarrow \epsilon$
2. $P \rightarrow 0$
3. $P \rightarrow 1$
4. $P \rightarrow 0P0$
5. $P \rightarrow 1P1$

the following is a parse tree:



It shows the derivation of $P \xRightarrow{*} 0110$.

The Yield of a Parse Tree

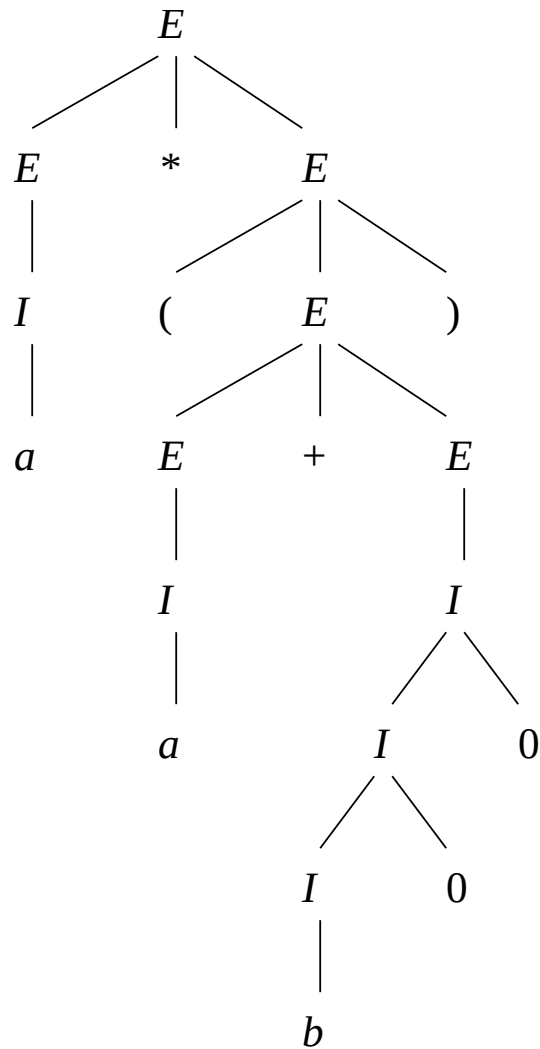
The *yield* of a parse tree is the string of leaves from left to right.

Important are those parse trees where:

1. The yield is a terminal string.
2. The root is labelled by the start symbol

We shall see that the set of yields of these important parse trees is the language of the grammar.

Example: Below is an important parse tree



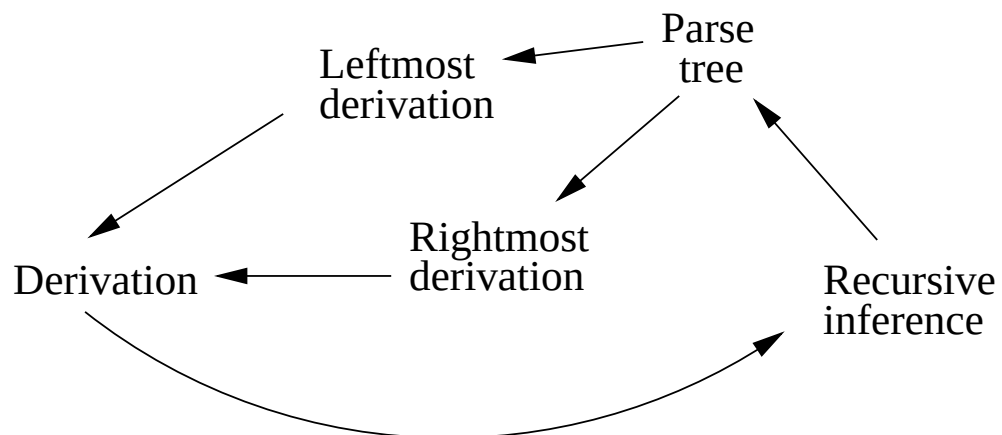
The yield is $a * (a + b00)$.

Compare the parse tree with the corresponding derivation in the expressions grammar.

Let $G = (V, T, P, S)$ be a CFG, and $A \in V$. We are going to show that the following are equivalent for a terminal string w :

1. We can determine by recursive inference that w is in the language of A
2. $A \xRightarrow{*} w$
3. $A \xRightarrow[lm]{*} w$
4. $A \xRightarrow[rm]{*} w$
5. There is a parse tree of G with root A and yield w .

To prove the equivalences, we use the following plan.

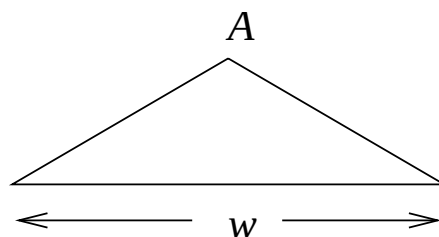


From Inferences to Trees

Theorem 5.12: Let $G = (V, T, P, S)$ be a CFG, and suppose we can show $w \in T^*$ to be in the language of a variable A . Then there is a parse tree for G with root A and yield w .

Proof: We do an induction on the length of the inference.

Basis: One step. Then we must have used a production $A \rightarrow w$. The desired parse tree is then



Induction: w is inferred to be in the language of A in $n + 1$ steps. Suppose the last step was based on a production

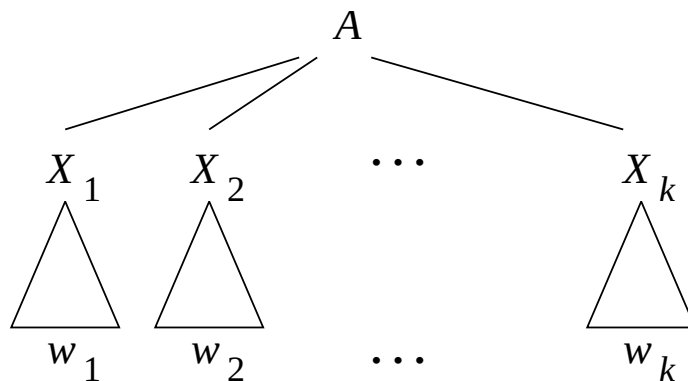
$$A \rightarrow X_1 X_2 \cdots X_k,$$

where $X_i \in V \cup T$. We break w up as

$$w_1 w_2 \cdots w_k,$$

where $w_i = X_i$, when $X_i \in T$, and when $X_i \in V$, then w_i was previously inferred being in the language of X_i , in at most n steps.

By the IH there are parse trees i with root X_i and yield w_i . Then the following is a parse tree for G with root A and yield w :



From trees to derivations

We'll show how to construct a leftmost derivation from a parse tree. First notice how one derivation of a string from a variable can be embedded within another derivation:

Example: In the expressions grammar there clearly is a derivation

$$E \Rightarrow I \Rightarrow Ib \Rightarrow ab.$$

Then, for any α and β there is a derivation

$$\alpha E \beta \Rightarrow \alpha I \beta \Rightarrow \alpha Ib \beta \Rightarrow \alpha ab \beta.$$

For example, suppose we have a derivation

$$E \Rightarrow E + E \Rightarrow E + (E).$$

Then we can choose $\alpha = E + ($ and $\beta =)$ and continue the derivation as

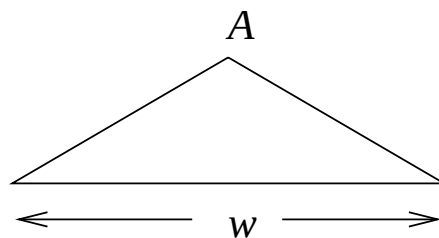
$$E + (E) \Rightarrow E + (I) \Rightarrow E + (Ib) \Rightarrow E + (ab).$$

This is why CFG's are called context-free.

Theorem 5.14: Let $G = (V, T, P, S)$ be a CFG, and suppose there is a parse tree with root labelled A and yield $w \in T^*$. Then $A \xRightarrow[lm]{*} w$ in G .

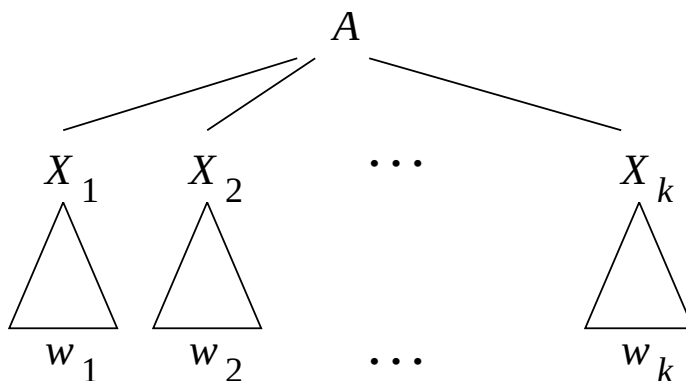
Proof: We do an induction on the height of the parse tree.

Basis: Height is 1. The tree must look like



Consequently $A \rightarrow w \in P$, and $A \xRightarrow[lm]{*} w$.

Induction: Height is $n + 1$. The tree must look like



Then $w = w_1 w_2 \cdots w_k$, where

1. If $X_i \in T$ then $w_i = X_i$.
2. If $X_i \in V$ then, as $w_i \in T^*$, $X_i \xrightarrow[lm]{*} w_i$ in G by the IH.

Now we construct $A \xRightarrow[lm]{*} w$ by an (inner) induction by showing that

$$\forall i : A \xRightarrow[lm]{*} w_1 w_2 \cdots w_i X_{i+1} X_{i+2} \cdots X_k.$$

Basis: Let $i = 0$. We already know that $A \xRightarrow[lm]{} X_1 X_2 \cdots X_k$.

Induction: Make the IH that

$$A \xRightarrow[lm]{*} w_1 w_2 \cdots w_{i-1} X_i X_{i+1} \cdots X_k.$$

As $w \in T^*$, also $w_1 w_2 \cdots w_{i-1} \in T^*$. So $w_1 w_2 \cdots w_{i-1}$ does not contain the leftmost variable.

(Case 1:) $X_i \in T$. Then $X_i = w_i$, which gives us

$$A \xRightarrow[lm]{*} w_1 w_2 \cdots w_i X_{i+1} \cdots X_k.$$

(Case 2:) $X_i \in V$. Then there is a derivation $X_i \Rightarrow_{lm} \alpha_1 \Rightarrow_{lm} \alpha_2 \Rightarrow_{lm} \cdots \Rightarrow_{lm} w_i$. By the context-free property of derivations we can proceed with

$$A \xRightarrow[lm]{*}$$

$$w_1 w_2 \cdots w_{i-1} X_i X_{i+1} \cdots X_k \Rightarrow_{lm}$$

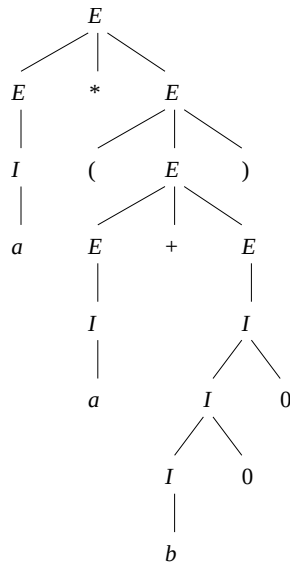
$$w_1 w_2 \cdots w_{i-1} \alpha_1 X_{i+1} \cdots X_k \Rightarrow_{lm}$$

$$w_1 w_2 \cdots w_{i-1} \alpha_2 X_{i+1} \cdots X_k \Rightarrow_{lm}$$

...

$$w_1 w_2 \cdots w_{i-1} w_i X_{i+1} \cdots X_k$$

Example: Let's construct the leftmost derivation for the tree



Suppose we have inductively constructed the leftmost derivation

$$E \underset{lm}{\Rightarrow} I \underset{lm}{\Rightarrow} a$$

corresponding to the leftmost subtree, and the leftmost derivation

$$\begin{array}{ccccccccc} E & \Rightarrow_{lm} & (E) & \Rightarrow_{lm} & (E + E) & \Rightarrow_{lm} & (I + E) & \Rightarrow_{lm} & (a + E) & \Rightarrow_{lm} \\ (a + I) & \Rightarrow_{lm} & (a + I0) & \Rightarrow_{lm} & (a + I00) & \Rightarrow_{lm} & (a + b00) & & & \end{array}$$

corresponding to the righmost subtree.

For the derivation corresponding to the whole tree we start with $E \Rightarrow_{lm} E * E$ and expand the first E with the first derivation and the second E with the second derivation:

$$\begin{aligned}
& E \Rightarrow_{lm} \\
& E * E \Rightarrow_{lm} \\
& I * E \Rightarrow_{lm} \\
& a * E \Rightarrow_{lm} \\
& a * (E) \Rightarrow_{lm} \\
& a * (E + E) \Rightarrow_{lm} \\
& a * (I + E) \Rightarrow_{lm} \\
& a * (a + E) \Rightarrow_{lm} \\
& a * (a + I) \Rightarrow_{lm} \\
& a * (a + I0) \Rightarrow_{lm} \\
& a * (a + I00) \Rightarrow_{lm} \\
& a * (a + b00)
\end{aligned}$$

From Derivations to Recursive Inferences

Observation: Suppose that $A \xRightarrow{*} X_1 X_2 \cdots X_k \xRightarrow{*} w$.
Then $w = w_1 w_2 \cdots w_k$, where $X_i \xRightarrow{*} w_i$. (To prove this see Exercise 5.2.4)

The factor w_i can be extracted from $A \xRightarrow{*} w$ by looking at the expansion of X_i only.

Example: $E \xRightarrow{*} a * b + a$, and

$$E \xRightarrow{*} \underbrace{E}_{X_1} \underbrace{*}_{X_2} \underbrace{E}_{X_3} \underbrace{+}_{X_4} \underbrace{E}_{X_5}$$

We have

$$\begin{aligned} E &\Rightarrow E * E \Rightarrow E * E + E \Rightarrow I * E + E \Rightarrow I * I + E \Rightarrow \\ &I * I + I \Rightarrow a * I + I \Rightarrow a * b + I \Rightarrow a * b + a \end{aligned}$$

By looking at the expansion of $X_3 = E$ only, we can extract

$$E \Rightarrow I \Rightarrow b$$

Theorem 5.18: Let $G = (V, T, P, S)$ be a CFG. Suppose $A \xRightarrow[G]{*} w$, where w is a string of *terminals*. Then we can infer that w is in the language of variable A .

Proof: We do an induction on the length of the derivation $A \xRightarrow[G]{*} w$.

Basis: One step. If $A \xRightarrow[G]{} w$ there must be a production $A \rightarrow w$ in P . Then we can infer that w is in the language of A .

Induction: Suppose $A \xRightarrow[G]{*} w$ in $n + 1$ steps. Write the derivation as

$$A \xRightarrow[G]{} X_1 X_2 \cdots X_k \xRightarrow[G]{*} w$$

Then as noted prior to the theorem we can break w as $w_1 w_2 \cdots w_k$ where $X_i \xRightarrow[G]{*} w_i$. Furthermore, $X_i \xRightarrow[G]{*} w_i$ can use at most n steps.

Now we have a production $A \rightarrow X_1 X_2 \cdots X_k$, and we know by the IH that we can infer w_i to be in the language of X_i or $w_i = X_i$.

By definition we can infer $w_1 w_2 \cdots w_k$ to be in the language of A .

Ambiguity in Grammars and Languages

In the expressions grammar

1. $E \rightarrow I$
2. $E \rightarrow E + E$
3. $E \rightarrow E * E$
4. $E \rightarrow (E)$
- ...

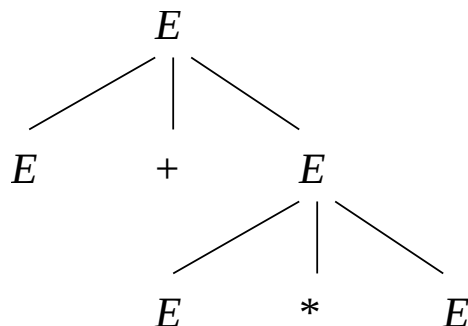
the sentential form $E + E * E$ has two derivations:

$$E \Rightarrow E + E \Rightarrow E + E * E$$

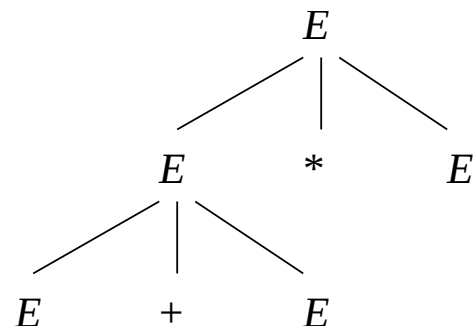
and

$$E \Rightarrow E * E \Rightarrow E + E * E$$

This gives us two parse trees, which implies an ambiguity:



(a)



(b)

The mere existence of several *derivations* is not dangerous, it is the existence of several parse trees that ruins a grammar.

Example: In the same grammar

- 5. $I \rightarrow a$
- 6. $I \rightarrow b$
- 7. $I \rightarrow Ia$
- 8. $I \rightarrow Ib$
- 9. $I \rightarrow I0$
- 10. $I \rightarrow I1$

the string $a + b$ has several derivations, e.g.

$$E \Rightarrow E + E \Rightarrow I + E \Rightarrow a + E \Rightarrow a + I \Rightarrow a + b$$

and

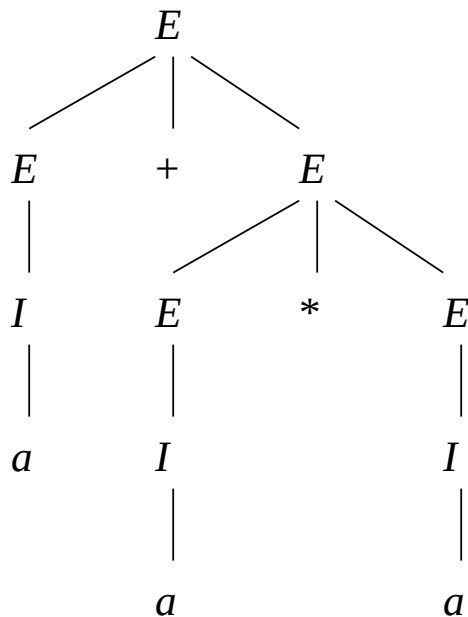
$$E \Rightarrow E + E \Rightarrow E + I \Rightarrow I + I \Rightarrow I + b \Rightarrow a + b$$

However, their parse trees are the same. The structure of $a + b$ is unambiguous.

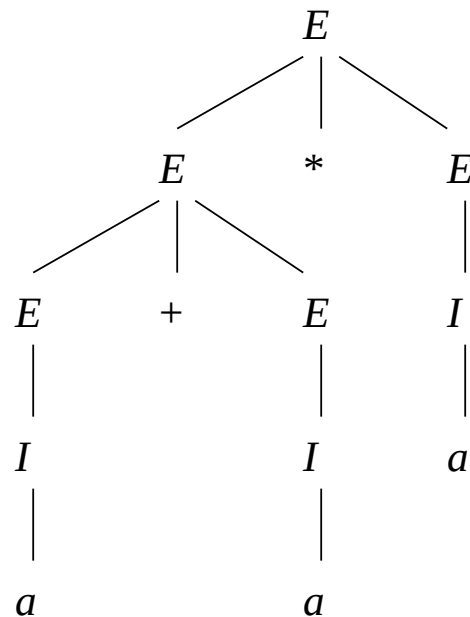
Definition: Let $G = (V, T, P, S)$ be a CFG. We say that G is *ambiguous* if there is a string in T^* that has more than one parse tree.

If every string in $L(G)$ has at most one parse tree, G is said to be *unambiguous*.

Example: The terminal string $a + a * a$ has two parse trees:



(a)



(b)

Removing Ambiguity From Grammars

Good news: Sometimes we can remove ambiguity “by hand”

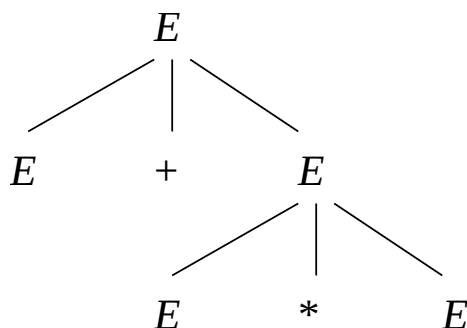
Bad news: There is no algorithm to do it

More bad news: Some CFL's have only ambiguous CFG's

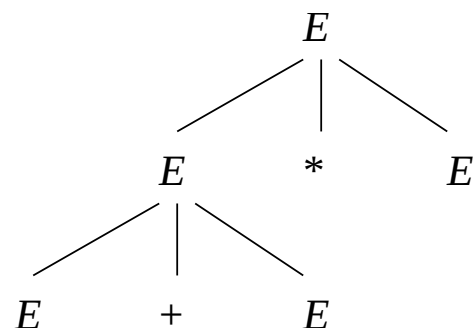
In the expressions grammar there are two problems:

1. There is no precedence between $*$ and $+$: we need to force the structure of figure (a).
2. There is no grouping of sequences of identical operators, e.g. is $E + E + E$ meant to be $E + (E + E)$ or $(E + E) + E$? The conventional approach is to insist on grouping from the left.

If the $*$'s in the following figure were replaced by $+$'s, then the structure of figure (b) is the only correct grouping.



(a)



(b)

Solution: We introduce more variables, each representing expressions of same “binding strength.”

1. A *factor* is an expression that cannot be broken apart by an adjacent $*$ or $+$. Our factors are

(a) Identifiers

(b) A parenthesized expression.

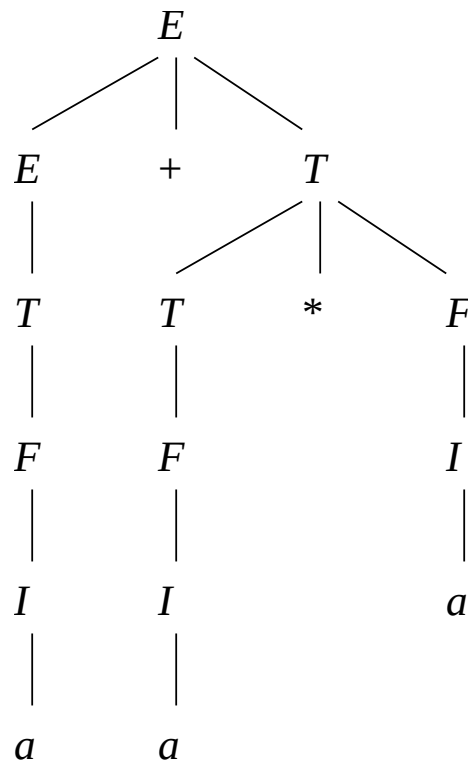
2. A *term* is an expression that cannot be broken by $+$. For instance $a * b$ can be broken by $a1*$. It cannot be broken by $+$, since e.g. $a1 + a * b$ is (by precedence rules) the same as $a1 + (a * b)$, and $a * b + a1$ is the same as $(a * b) + a1$.

3. An *expression* refers to any possible expression, including those that can be broken by an adjacent $*$ or $+$.

We'll let F stand for factors, T for terms, and E for expressions. Consider the following grammar:

1. $I \rightarrow a \mid b \mid Ia \mid Ib \mid IO \mid I1$
2. $F \rightarrow I \mid (E)$
3. $T \rightarrow F \mid T * F$
4. $E \rightarrow T \mid E + T$

Now the only parse tree for $a + a * a$ will be



Why is the new grammar unambiguous?

Intuitive explanation:

- A factor is either an identifier or (E) , for some expression E .
- A term is a sequence

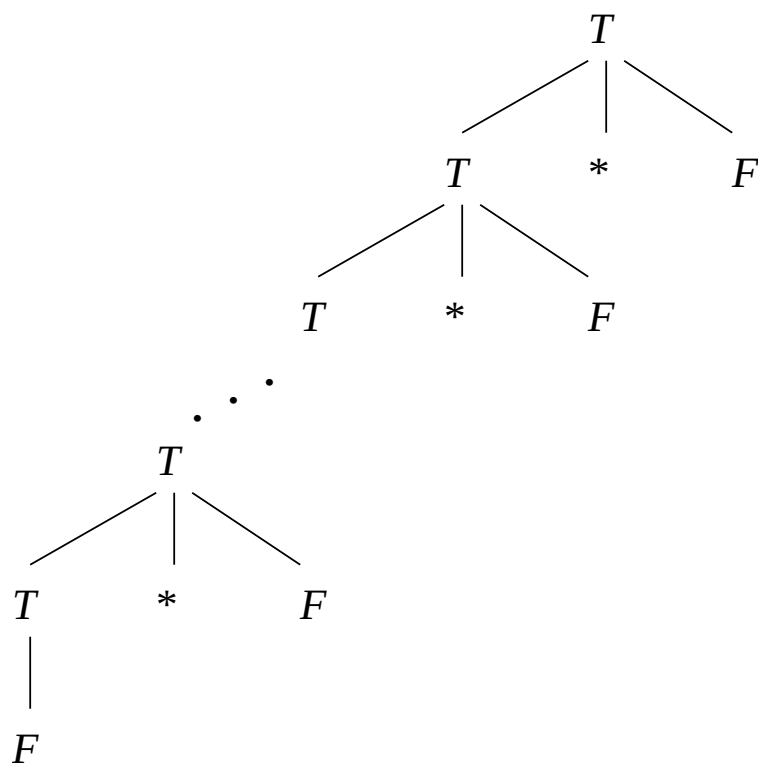
$$f_1 * f_2 * \cdots * f_{n-1} * f_n$$

of factors. The only parse tree is the one that gives $f_1 * f_2 * \cdots * f_{n-1}$ as a term and f_n as a factor, as in the parse tree on the next slide.

- An expression is a sequence

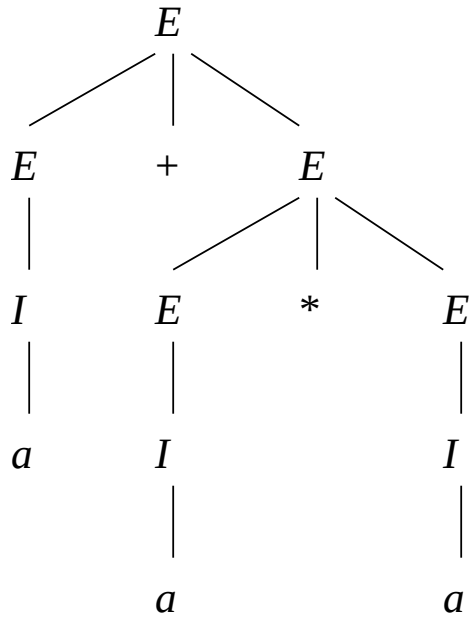
$$t_1 + t_2 + \cdots + t_{n-1} + t_n$$

of terms t_i . It can only be parsed with $t_1 + t_2 + \cdots + t_{n-1}$ as an expression and t_n as a term.

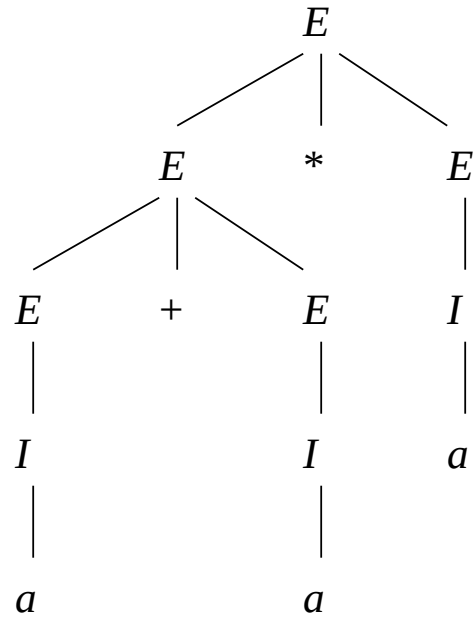


Leftmost derivations and Ambiguity

The two parse trees for $a + a * a$



(a)



(b)

give rise to two derivations:

$$\begin{aligned}
 E &\Rightarrow_{lm} E + E \Rightarrow_{lm} I + E \Rightarrow_{lm} a + E \Rightarrow_{lm} a + E * E \\
 &\Rightarrow_{lm} a + I * E \Rightarrow_{lm} a + a * E \Rightarrow_{lm} a + a * I \Rightarrow_{lm} a + a * a
 \end{aligned}$$

and

$$\begin{aligned}
 E &\Rightarrow_{lm} E * E \Rightarrow_{lm} E + E * E \Rightarrow_{lm} I + E * E \Rightarrow_{lm} a + E * E \\
 &\Rightarrow_{lm} a + I * E \Rightarrow_{lm} a + a * E \Rightarrow_{lm} a + a * I \Rightarrow_{lm} a + a * a
 \end{aligned}$$

In general:

- One parse tree may imply many derivations.
- Many *leftmost* derivations imply many parse trees.
- Many *rightmost* derivations imply many parse trees.

Theorem 5.29: For any CFG G , a terminal string w has two distinct parse trees if and only if w has two distinct leftmost derivations from the start symbol.

Proof: (*Only If.*) If the two parse trees differ, they have a node at which different productions are used, say $A \rightarrow X_1X_2\cdots X_k$ and $A \rightarrow Y_1Y_2\cdots Y_m$. The corresponding leftmost derivations will use derivations based on these two different productions and will thus be distinct.

(*If.*) Let's look at how we construct a parse tree from a leftmost derivation. If there are two distinct derivations, then at the first step where the derivations differ, the corresponding nodes in the two parse trees being constructed will get different lists of children. It should now be clear that two distinct derivations give rise to two different parse trees.

Inherent Ambiguity

A CFL L is *inherently ambiguous* if *all* grammars for L are ambiguous.

Example: Consider $L =$

$$\{a^n b^n c^m d^m : n \geq 1, m \geq 1\} \cup \{a^n b^m c^m d^n : n \geq 1, m \geq 1\}.$$

A grammar for L is

$$S \rightarrow AB \mid C$$

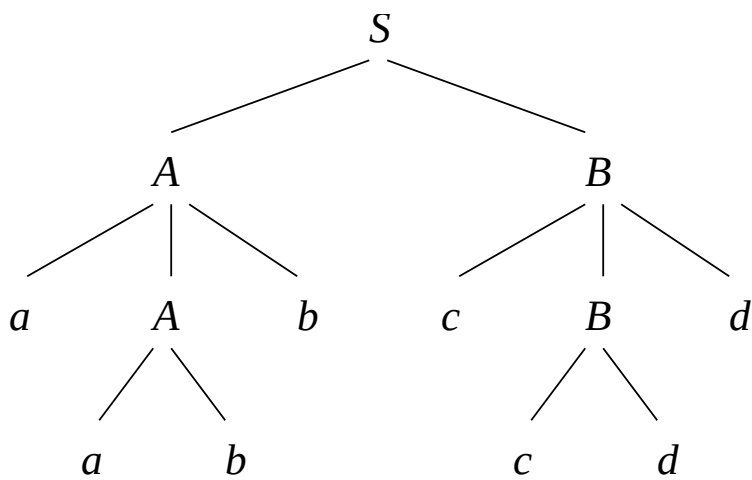
$$A \rightarrow aAb \mid ab$$

$$B \rightarrow cBd \mid cd$$

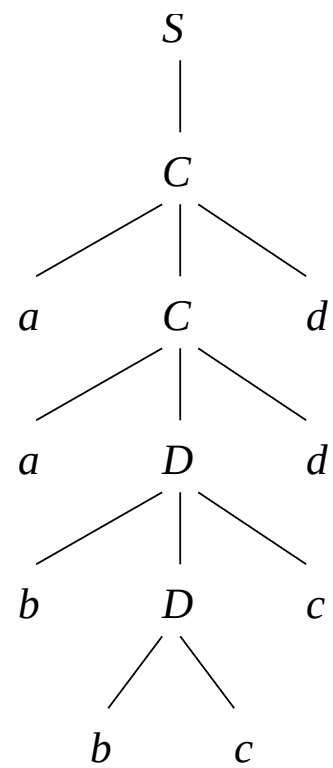
$$C \rightarrow aCd \mid aDd$$

$$D \rightarrow bDc \mid bc$$

Let's look at parsing the string *aabbccdd*.



(a)



(b)

From this we see that there are two leftmost derivations:

$$S \underset{lm}{\Rightarrow} AB \underset{lm}{\Rightarrow} aAbB \underset{lm}{\Rightarrow} aabbB \underset{lm}{\Rightarrow} aabbcBd \underset{lm}{\Rightarrow} aabbccdd$$

and

$$S \underset{lm}{\Rightarrow} C \underset{lm}{\Rightarrow} aCd \underset{lm}{\Rightarrow} aaDdd \underset{lm}{\Rightarrow} aabDcdd \underset{lm}{\Rightarrow} aabbccdd$$

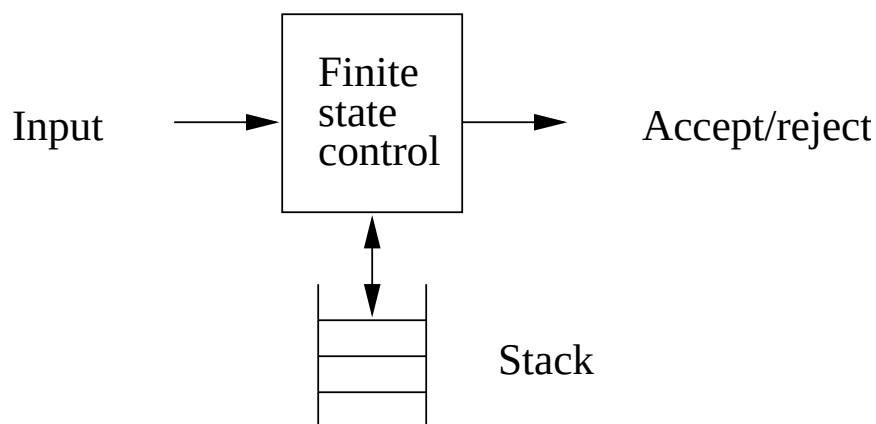
It can be shown that every grammar for L behaves like the one above. The language L is inherently ambiguous.

Pushdown Automata

A pushdown automata (PDA) is essentially an ϵ -NFA with a stack.

On a transition the PDA:

1. Consumes an input symbol.
2. Goes to a new state (or stays in the old).
3. Replaces the top of the stack by any string (does nothing, pops the stack, or pushes a string onto the stack)



Example: Let's consider

$$L_{ww^R} = \{ww^R : w \in \{0,1\}^*\},$$

with “grammar” $P \rightarrow 0P0$, $P \rightarrow 1P1$, $P \rightarrow \epsilon$.

A PDA for L_{ww^R} has three states, and operates as follows:

1. Guess that you are reading w . Stay in state q_0 , and push the input symbol onto the stack.
2. Guess that you're in the middle of ww^R . Go spontaneously to state q_1 .
3. You're now reading the head of w^R . Compare it to the top of the stack. If they match, pop the stack, and remain in state q_1 . If they don't match, go to sleep.
4. If the stack is empty, go to state q_2 and accept.

PDA formally

A PDA is a seven-tuple:

$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F),$$

where

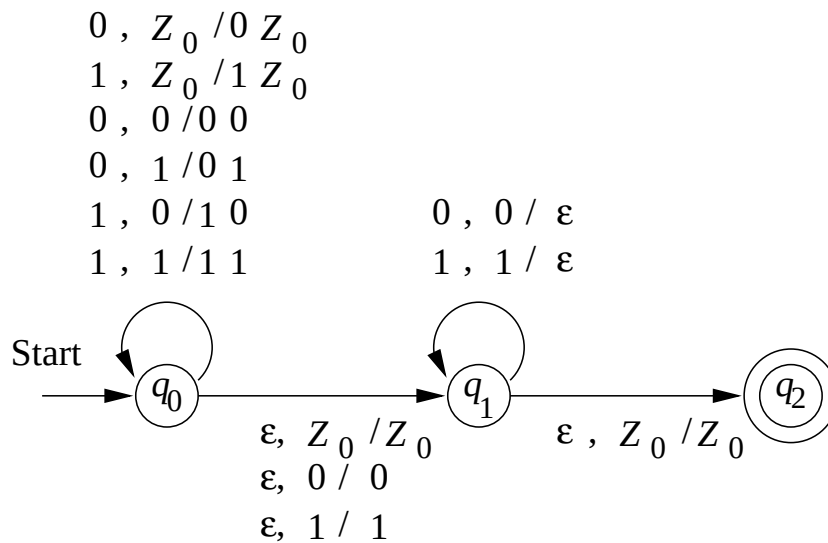
- Q is a finite set of states,
- Σ is a finite *input alphabet*,
- Γ is a finite *stack alphabet*,
- q_0 is the *start state*,
- $Z_0 \in \Gamma$ is the *start symbol* for the stack: initially the stack contains exactly one Z_0 and nothing else,
- $F \subseteq Q$ is the set of *accepting states*,
- $\delta : Q \times \Sigma \cup \{\epsilon\} \times \Gamma \rightarrow 2^{Q \times \Gamma^*}$ is the *transition function*. If $(p, \gamma) \in \delta(q, a, X)$ then X is popped from the stack and the symbols of the string γ are pushed one by one in reverse order on the stack (the leftmost symbol becomes the new top).

Note that if $\gamma = \epsilon$, the stack is popped, if $\gamma = X$ the stack is unchanged. If $\gamma = YZ$, then X is replaced by Z , and Y is pushed onto the stack.

In a *transition diagram* for a PDA:

- the nodes correspond to the states of the PDA,
- an arrow labeled *Start* indicates the start state,
- doubly circled states indicate accepting states,
- an arc from q to p , labeled $a, X/\alpha$, corresponds to the transition $(p, \alpha) \in \delta(q, a, X)$. Thus the arc label indicates what input is used, and the old and new tops of the stack.

Example: The PDA for L_{wwr} as a transition diagram:



is actually the seven-tuple

$$P = (\{q_0, q_1, q_2\}, \{0, 1\}, \{0, 1, Z_0\}, \delta, q_0, Z_0, \{q_2\}),$$

where δ is given by the following table (set brackets missing):

	$0, Z_0$	$1, Z_0$	$0, 0$	$0, 1$	$1, 0$	$1, 1$	ϵ, Z_0	$\epsilon, 0$	$\epsilon, 1$
$\rightarrow q_0$	$q_0, 0Z_0$	$q_0, 1Z_0$	$q_0, 00$	$q_0, 01$	$q_0, 10$	$q_0, 11$	q_1, Z_0	$q_1, 0$	$q_1, 1$
q_1			q_1, ϵ			q_1, ϵ	q_2, Z_0		
$\star q_2$									

Instantaneous Descriptions

A PDA goes from configuration to configuration when consuming input.

To reason about PDA computation, we use *instantaneous descriptions* of the PDA. An ID is a triple

$$(q, w, \gamma)$$

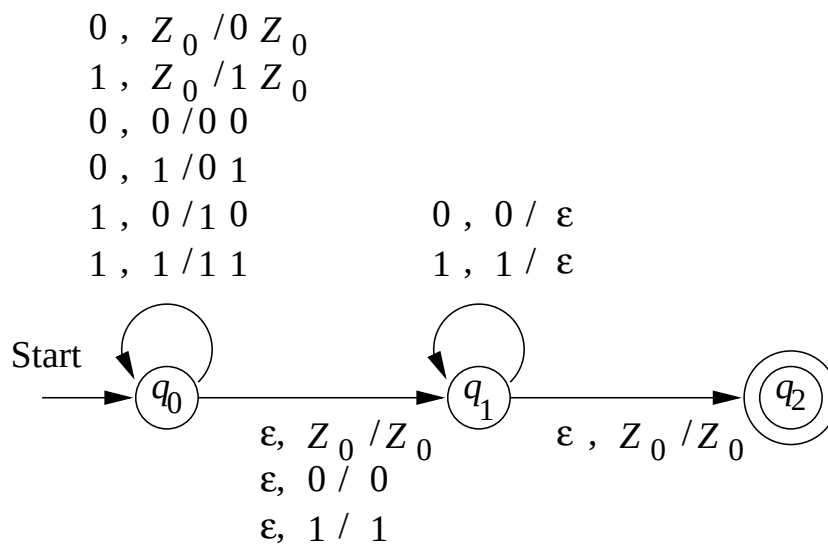
where q is the state, w the remaining input, and γ the stack contents ($\gamma = \text{top} \cdots \text{bottom}$).

Definition $\vdash$: Let $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ be a PDA. Then $\forall w \in \Sigma^*, \beta \in \Gamma^*$:

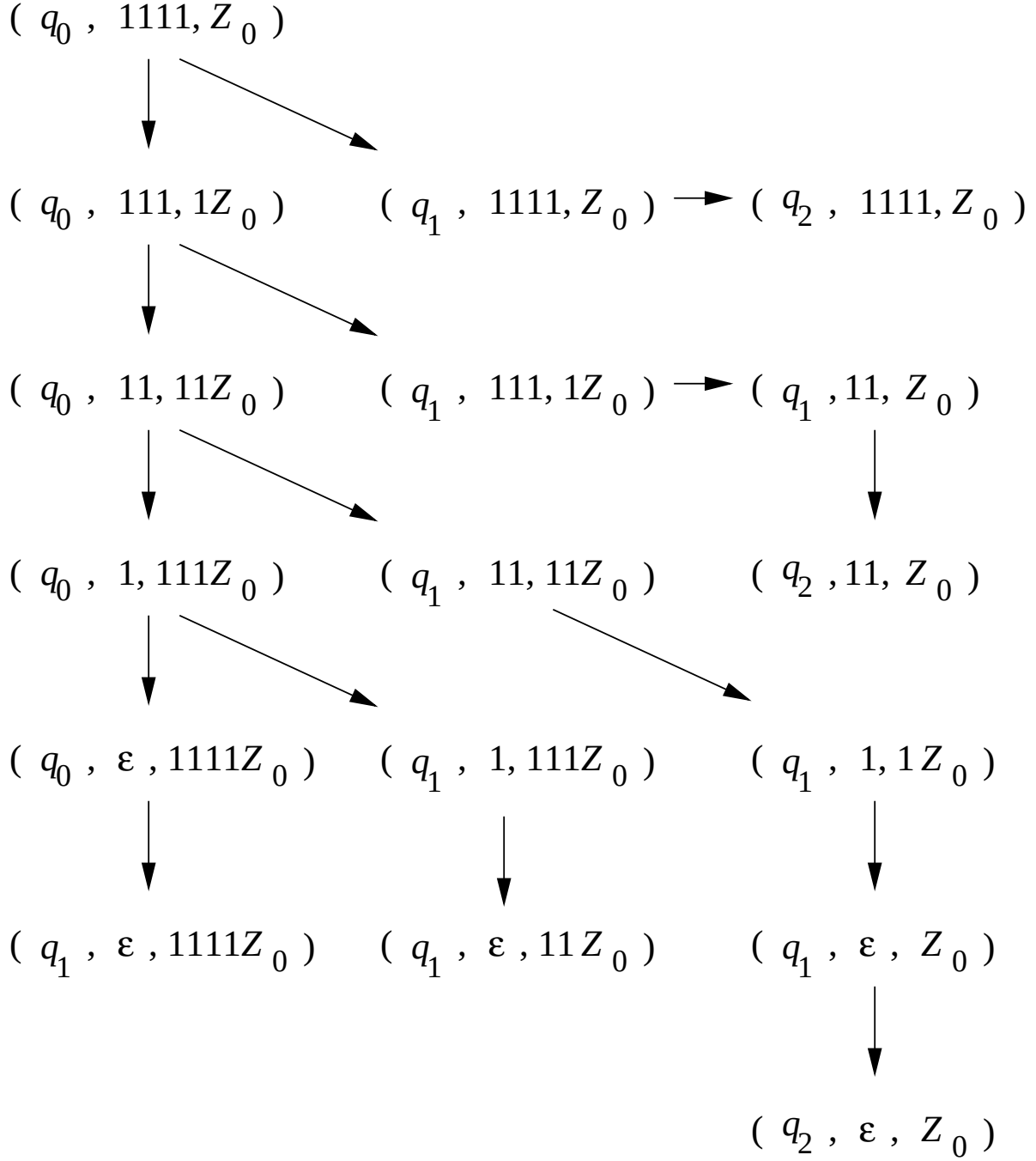
$$(p, \alpha) \in \delta(q, a, X) \Rightarrow (q, aw, X\beta) \vdash (p, w, \alpha\beta).$$

We define $\vdash^*$ to be the reflexive-transitive closure of $\vdash$.

Example: On input 1111 the PDA



has the following computation sequences, where arrows represent the $\vdash$ -relation:



The following properties about ID's hold:

1. If an ID sequence is a legal computation for a PDA, then so is the sequence obtained by adding an additional string at the end of component number two.
2. If an ID sequence is a legal computation for a PDA, then so is the sequence obtained by adding an additional string at the bottom of component number three.
3. If an ID sequence is a legal computation for a PDA, and some tail of the input is not consumed, then removing this tail from all IDs results in a legal computation sequence.

Theorem 6.5: $\forall w \in \Sigma^*, \gamma \in \Gamma^* :$

$$(q, x, \alpha) \vdash^* (p, y, \beta) \Rightarrow (q, xw, \alpha\gamma) \vdash^* (p, yw, \beta\gamma).$$

Proof: Induction on the length of the sequence to the left.

Note: If $\gamma = \epsilon$ we have property 1, and if $w = \epsilon$ we have property 2.

Note2: The reverse of the theorem is false.

For property 3 we have

Theorem 6.6:

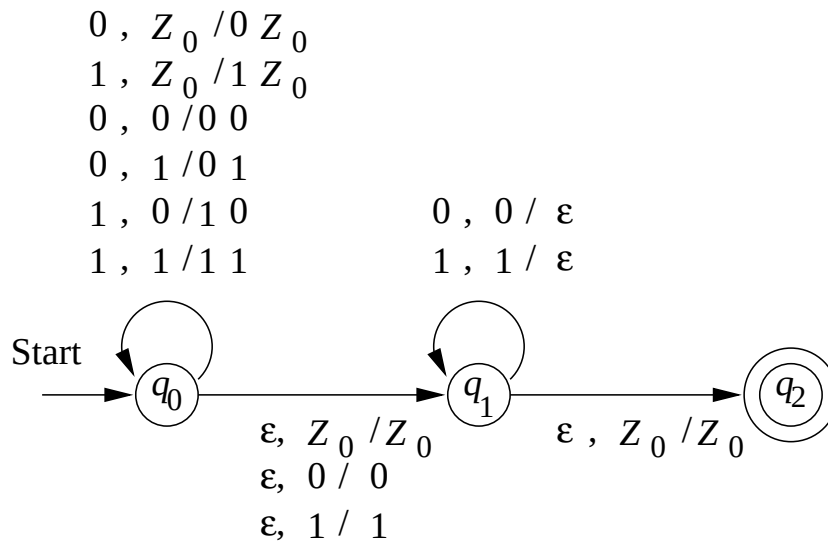
$$(q, xw, \alpha) \vdash^* (p, yw, \beta) \Rightarrow (q, x, \alpha) \vdash^* (p, y, \beta).$$

Acceptance by final state

Let $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ be a PDA. The language accepted by P by final state is

$$L(P) = \{w : (q_0, w, Z_0) \vdash^* (q, \epsilon, \alpha), q \in F\}.$$

Example: Reconsider the PDA P



P accepts exactly L_{wwr} by final state.

We prove this by showing that $L(P) = L_{ww^R}$: P accepts the string x by final state if and only if x is of the form ww^R .

Proof

($\supseteq$ -direction.) Let $x \in L_{ww^R}$. Then $x = ww^R$, and the following is a legal computation sequence

$$(q_0, ww^R, Z_0) \vdash^* (q_0, w^R, w^R Z_0) \vdash (q_1, w^R, w^R Z_0) \vdash^* \\ (q_1, \epsilon, Z_0) \vdash (q_2, \epsilon, Z_0).$$

($\subseteq$ -direction.)

Observe that the only way the PDA can enter q_2 is if it is in state q_1 with Z_0 at the top of the stack.

Thus it is sufficient to show that if $(q_0, x, Z_0) \vdash^* (q_1, \epsilon, Z_0)$ then $x = ww^R$, for some word w .

We'll show by induction on $|x|$ that

$$(q_0, x, \alpha) \vdash^* (q_1, \epsilon, \alpha) \Rightarrow x = ww^R.$$

Basis: If $x = \epsilon$ then x is a palindrome.

Induction: Suppose $x = a_1a_2 \cdots a_n$, where $n > 0$, and the IH holds for shorter strings.

There are two moves for the PDA from ID (q_0, x, α) :

Move 1: The spontaneous $(q_0, x, \alpha) \vdash (q_1, x, \alpha)$.
 Now $(q_1, x, \alpha) \vdash^* (q_1, \epsilon, \beta)$ implies that $|\beta| < |\alpha|$
 (in q_1 we can only pop) and $\beta \neq \alpha$, and there-
 fore the condition $(q_0, x, \alpha) \vdash^* (q_1, \epsilon, \alpha)$ cannot
 be satisfied.

Move 2: Loop and push $(q_0, a_1 a_2 \cdots a_n, \alpha) \vdash$
 $(q_0, a_2 \cdots a_n, a_1 \alpha)$.

In this case the only way a sequence of moves
 can end in (q_1, ϵ, α) is:

$(q_0, a_1 a_2 \cdots a_n, \alpha) \vdash (q_0, a_2 \cdots a_n, a_1 \alpha) \vdash \cdots \vdash$
 $(q_1, a_n, a_1 \alpha) \vdash (q_1, \epsilon, \alpha)$.

The last move is a pop and $a_1 = a_n$. In

$(q_0, a_2 \cdots a_n, a_1 \alpha) \vdash^* (q_1, a_n, a_1 \alpha)$.

we can remove a_n by Theorem 6.6:

$(q_0, a_2 \cdots a_{n-1}, a_1 \alpha) \vdash^* (q_1, \epsilon, a_1 \alpha)$.

Then, by the IH $a_2 \cdots a_{n-1} = yy^R$ and $x =$
 $a_1 yy^R a_n$ is a palindrome.

Acceptance by Empty Stack

Let $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ be a PDA. The *language accepted by P by empty stack* is

$$N(P) = \{w : (q_0, w, Z_0) \vdash^* (q, \epsilon, \epsilon)\}.$$

Note: q can be any state and F is optional.

Question: How to modify the palindrome-PDA to accept by empty stack?

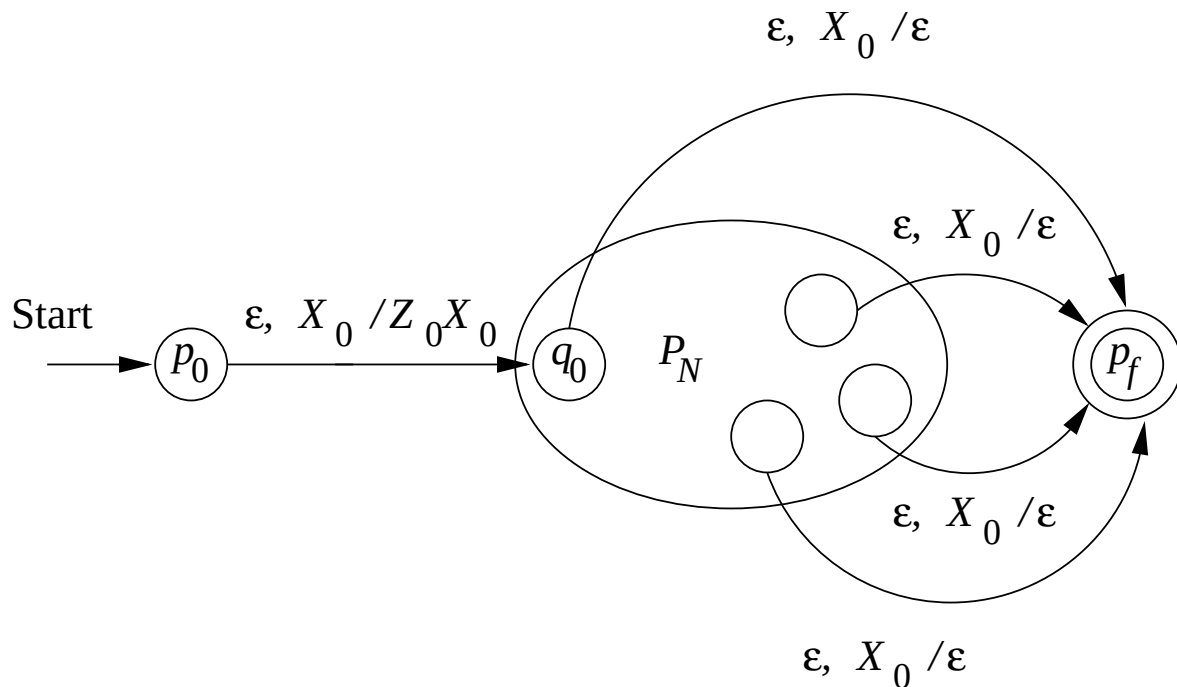
From Empty Stack to Final State

Theorem 6.9: If $L = N(P_N)$ for some PDA $P_N = (Q, \Sigma, \Gamma, \delta_N, q_0, Z_0)$, then $\exists$ PDA P_F , such that $L = L(P_F)$.

Proof: Let

$$P_F = (Q \cup \{p_0, p_f\}, \Sigma, \Gamma \cup \{X_0\}, \delta_F, p_0, X_0, \{p_f\})$$

where $\delta_F(p_0, \epsilon, X_0) = \{(q_0, Z_0 X_0)\}$, and for all $q \in Q, a \in \Sigma \cup \{\epsilon\}, Y \in \Gamma : \delta_F(q, a, Y) = \delta_N(q, a, Y)$, and in addition $(p_f, \epsilon) \in \delta_F(q, \epsilon, X_0)$.



We have to show that $L(P_F) = N(P_N)$.

($\supseteq$ direction.) Let $w \in N(P_N)$. Then

$$(q_0, w, Z_0) \vdash_N^* (q, \epsilon, \epsilon),$$

for some q . From Theorem 6.5 we get

$$(q_0, w, Z_0 X_0) \vdash_N^* (q, \epsilon, X_0).$$

Since $\delta_N \subset \delta_F$ we have

$$(q_0, w, Z_0 X_0) \vdash_F^* (q, \epsilon, X_0).$$

We conclude that

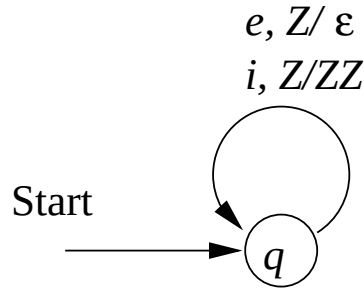
$$(p_0, w, X_0) \vdash_F (q_0, w, Z_0 X_0) \vdash_F^* (q, \epsilon, X_0) \vdash_F (p_f, \epsilon, \epsilon).$$

($\subseteq$ direction.) By inspecting the diagram.

Let's design P_N for catching *errors* in strings meant to be in the *if-else*-grammar G

$$S \rightarrow \epsilon | SS | iS | iSe.$$

Here $\{ieie, iie, iei\} \subseteq L(G)$, and $\{ei, ieeii\} \cap L(G) = \emptyset$.
The diagram for P_N is



Formally,

$$P_N = (\{q\}, \{i, e\}, \{Z\}, \delta_N, q, Z),$$

where $\delta_N(q, i, Z) = \{(q, ZZ)\}$,

and $\delta_N(q, e, Z) = \{(q, \epsilon)\}$.

From P_N we can construct

$$P_F = (\{p, q, r\}, \{i, e\}, \{Z, X_0\}, \delta_F, p, X_0, \{r\}),$$

where

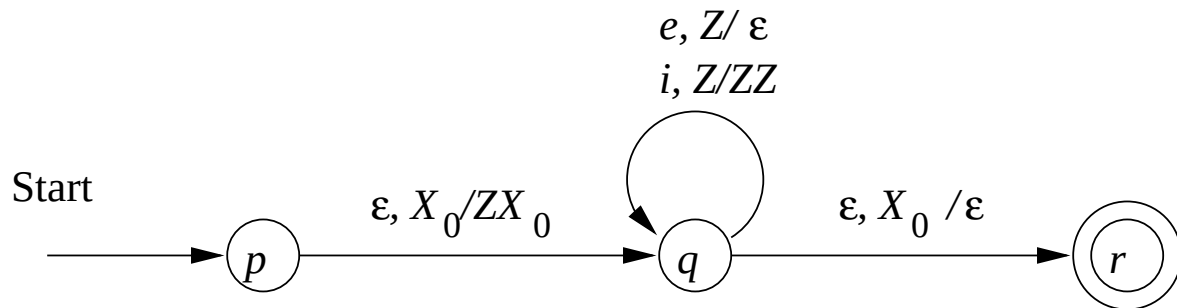
$$\delta_F(p, \epsilon, X_0) = \{(q, ZX_0)\},$$

$$\delta_F(q, i, Z) = \delta_N(q, i, Z) = \{(q, ZZ)\},$$

$$\delta_F(q, e, Z) = \delta_N(q, e, Z) = \{(q, \epsilon)\}, \text{ and}$$

$$\delta_F(q, \epsilon, X_0) = \{(r, \epsilon)\}$$

The diagram for P_F is



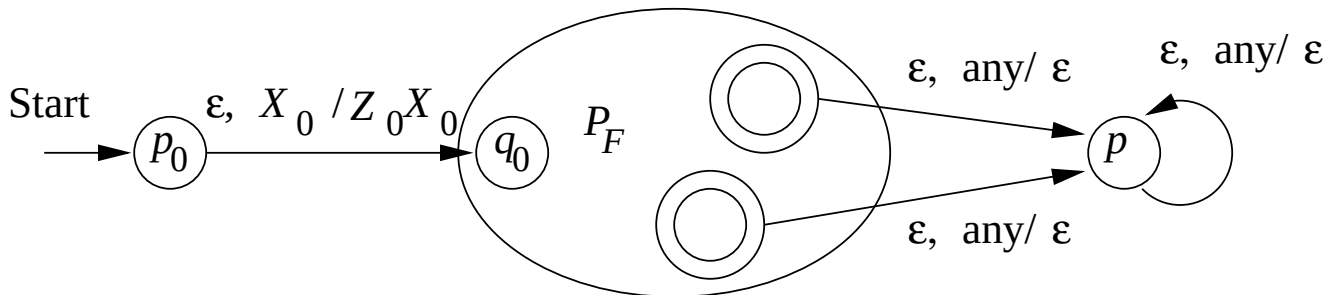
From Final State to Empty Stack

Theorem 6.11: Let $L = L(P_F)$, for some PDA $P_F = (Q, \Sigma, \Gamma, \delta_F, q_0, Z_0, F)$. Then $\exists$ PDA P_N , such that $L = N(P_N)$.

Proof: Let

$$P_N = (Q \cup \{p_0, p\}, \Sigma, \Gamma \cup \{X_0\}, \delta_N, p_0, X_0)$$

where $\delta_N(p_0, \epsilon, X_0) = \{(q_0, Z_0 X_0)\}$, $\delta_N(p, \epsilon, Y) = \{(p, \epsilon)\}$, for $Y \in \Gamma \cup \{X_0\}$, and for all $q \in Q$, $a \in \Sigma \cup \{\epsilon\}$, $Y \in \Gamma$: $\delta_N(q, a, Y) = \delta_F(q, a, Y)$, and in addition $\forall q \in F$, and $Y \in \Gamma \cup \{X_0\}$: $(p, \epsilon) \in \delta_N(q, \epsilon, Y)$.



We have to show that $N(P_N) = L(P_F)$.

($\subseteq$ -direction.) By inspecting the diagram.

($\supseteq$ -direction.) Let $w \in L(P_F)$. Then

$$(q_0, w, Z_0) \vdash_F^* (q, \epsilon, \alpha),$$

for some $q \in F, \alpha \in \Gamma^*$. Since $\delta_F \subseteq \delta_N$, and Theorem 6.5 says that X_0 can be slid under the stack, we get

$$(q_0, w, Z_0 X_0) \vdash_N^* (q, \epsilon, \alpha X_0).$$

Then P_N can compute:

$$(p_0, w, X_0) \vdash_N (q_0, w, Z_0 X_0) \vdash_N^* (q, \epsilon, \alpha X_0) \vdash_N^* (p, \epsilon, \epsilon).$$

Equivalence of PDA's and CFG's

A language is

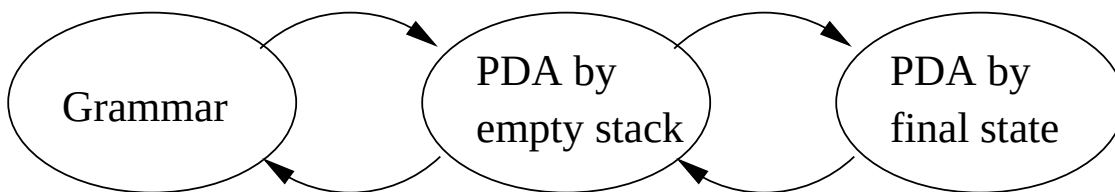
generated by a CFG

if and only if it is

accepted by a PDA by empty stack

if and only if it is

accepted by a PDA by final state



We already know how to go between null stack and final state.

From CFG's to PDA's

Given G , we construct a PDA that simulates $\xRightarrow{*}_{lm}$.

Any left-sentential form that is not a terminal string can be written as

$$xA\alpha$$

where A is the leftmost variable in the form. $A\alpha$ is called the tail. If a left-sentential form is a terminal string, then its tail is ϵ .

For instance,

$$\underbrace{(a+}_{x} \underbrace{\overbrace{E}^A}_{\alpha})_{\text{tail}}$$

Let $xA\alpha \xRightarrow{lm} x\beta\alpha$. This corresponds to the PDA first having consumed x and having $A\alpha$ on the stack, and then on ϵ it pops A and pushes β .

More formally, let y , such that $w = xy$. Then the PDA goes non-deterministically from configuration $(q, y, A\alpha)$ to configuration $(q, y, \beta\alpha)$.

At $(q, y, \beta\alpha)$ the PDA behaves as before, unless there are terminals in the prefix of β . In that case, the PDA pops them, provided it can consume matching input.

If all guesses are right, the PDA ends up with empty stack and input.

Formally, let $G = (V, T, Q, S)$ be a CFG. Define P_G as

$$(\{q\}, T, V \cup T, \delta, q, S),$$

where

$$\delta(q, \epsilon, A) = \{(q, \beta) : A \rightarrow \beta \in Q\},$$

for $A \in V$, and

$$\delta(q, a, a) = \{(q, \epsilon)\},$$

for $a \in T$.

See example 6.12 in your book.

Theorem 6.13: $N(P_G) = L(G)$.

Proof:

($\supseteq$ -direction.) Let $w \in L(G)$. Then

$$S = \gamma_1 \xRightarrow{lm} \gamma_2 \xRightarrow{lm} \cdots \xRightarrow{lm} \gamma_n = w$$

Let $\gamma_i = x_i \alpha_i$ where α_i is the tail of γ_i . We show by induction on i that

$$(q, w, S) \vdash^* (q, y_i, \alpha_i),$$

where y_i is the remaining input: $w = x_i y_i$.

Basis: For $i = 1$, $\gamma_1 = S$. Thus $x_1 = \epsilon$, and $y_1 = w$. Clearly $(q, w, S) \vdash^* (q, w, S)$.

Induction: IH is $(q, w, S) \vdash^* (q, y_i, \alpha_i)$. We have to show that

$$(q, y_i, \alpha_i) \vdash^* (q, y_{i+1}, \alpha_{i+1})$$

As $\gamma_i \xRightarrow{lm} \gamma_{i+1}$ and $\gamma_i = x_i A \chi$ there is a rule $A \rightarrow \beta$ and by the construction of the transition function of P_G , we first replace A at the top of the stack

$$(q, y_i, A\chi) \vdash (q, y_i, \beta\chi)$$

and then match any terminals on the top of the stack with the input symbols, giving

$$(q, y_{i+1}, \alpha_{i+1})$$

(note that $\gamma_{i+1} = x_i \beta \chi = x_{i+1} \alpha_{i+1}$)

To complete the proof, we note that $\gamma_n = w$, so the tail $\alpha_n = \epsilon$, $x_n = w$ and $y_n = \epsilon$, and thus $(q, w, S) \vdash^* (q, \epsilon, \epsilon)$, i.e. $w \in N(P_G)$.

($\subseteq$ -direction.) We shall show by an induction on the length of $\vdash^*$, that

(♣) If $(q, x, A) \vdash^* (q, \epsilon, \epsilon)$, then $A \xRightarrow{*} x$.

Basis: Length 1. Then by the construction of P_G it must be that $A \rightarrow \epsilon$ is in G , and $(q, \epsilon) \in \delta(q, \epsilon, A)$. Thus $A \xRightarrow{*} \epsilon$.

Induction: Length is $n > 1$, and the IH holds for lengths $< n$.

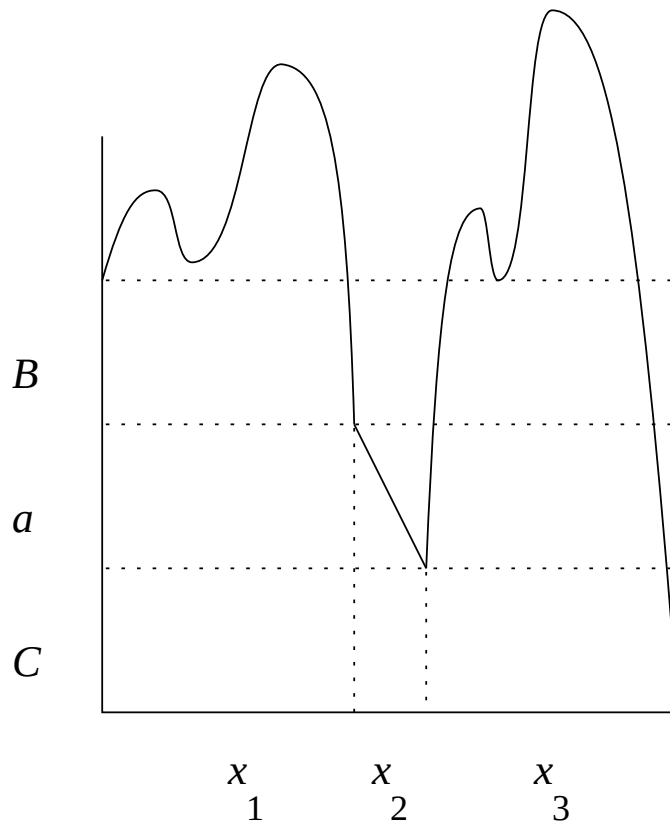
Since A is a variable, we must have

$$(q, x, A) \vdash (q, x, Y_1 Y_2 \cdots Y_k) \vdash \cdots (*) \cdots \vdash (q, \epsilon, \epsilon)$$

where $A \rightarrow Y_1 Y_2 \cdots Y_k$ is in G . At some time in $(*)$ first Y_2 , then Y_3 , then Y_4 , ... will be top of stack successively.

We can now write x as $x_1x_2\cdots x_k$, where x_1 is the input consumed until Y_1 is popped from the stack, then x_2 is the next input consumed until Y_2 is popped from the stack, and so on.

Example: In the figure below, $Y_1 = B$, $Y_2 = a$, and $Y_3 = C$.



Now we can conclude that

$$\forall i \in \{1, \dots, k\} : (q, x_i x_{i+1} \cdots x_k, Y_i) \vdash^* (q, x_{i+1} \cdots x_k, \epsilon)$$

in less than n steps and by Theorem 6.6

$$\forall i \in \{1, \dots, k\} : (q, x_i, Y_i) \vdash^* (q, \epsilon, \epsilon)$$

in less than n steps.

If Y_i is a variable we have by the IH that

$$Y_i \xRightarrow{*} x_i$$

If Y_i is a terminal, we have $|x_i| = 1$, and $Y_i = x_i$.

Thus $Y_i \xRightarrow{*} x_i$ by the reflexivity of $\xRightarrow{*}$.

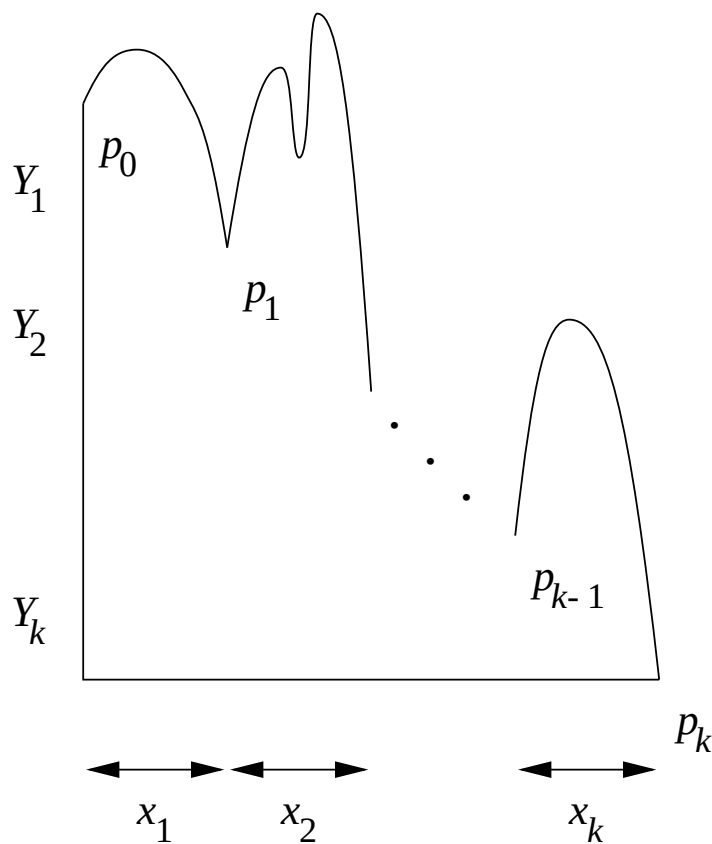
So we have

$$A \Rightarrow Y_1 Y_2 \cdots Y_k \xRightarrow{*} x_1 Y_2 \cdots Y_k \xRightarrow{*} \cdots \xRightarrow{*} x_1 x_2 \cdots x_k = x$$

The claim of the theorem now follows by choosing $A = S$, and $x = w$. Suppose $w \in N(P)$. Then $(q, w, S) \vdash^* (q, \epsilon, \epsilon)$, and by ($\clubsuit$), we have $S \xRightarrow{*} w$, meaning $w \in L(G)$.

From PDA's to CFG's

Let's look at how a PDA can consume $x = x_1x_2\cdots x_k$ and empty the stack.



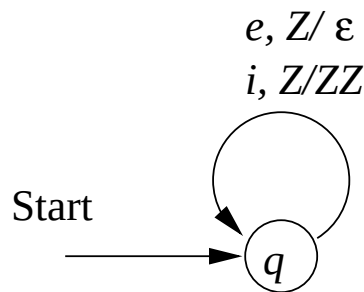
We shall define a grammar with variables of the form $[p_{i-1}Y_ip_i]$ representing going from p_{i-1} to p_i with net effect of popping Y_i .

Formally, let $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0)$ be a PDA.
Define $G = (V, \Sigma, R, S)$, where

$$\begin{aligned}
V &= \{[pXq] : \{p, q\} \subseteq Q, X \in \Gamma\} \cup \{S\} \\
R &= \{S \rightarrow [q_0Z_0p] : p \in Q\} \cup \\
&\quad \{[\mathbf{q}Xr_k] \rightarrow a[\mathbf{r}Y_1r_1][\mathbf{r}_1Y_2r_2] \cdots [r_{k-1}Y_kr_k] : \\
&\quad \quad a \in \Sigma \cup \{\epsilon\}, \\
&\quad \quad \{r_1, \dots, r_k\} \subseteq Q, \\
&\quad \quad (\mathbf{r}, Y_1Y_2 \cdots Y_k) \in \delta(\mathbf{q}, a, X)\} \\
&\cup \{[\mathbf{q}Xr] \rightarrow a : a \in \Sigma \cup \{\epsilon\}, \\
&\quad (\mathbf{r}, \epsilon) \in \delta(\mathbf{q}, a, X)\}
\end{aligned}$$

Example:

Let's convert $P_N = (\{q\}, \{i, e\}, \{Z\}, \delta_N, q, Z)$ where $\delta_N(q, i, Z) = \{(q, ZZ)\}$ and $\delta_N(q, e, Z) = \{(q, \epsilon)\}$



to a grammar

$$G = (V, \{i, e\}, R, S),$$

where $V = \{[qZq], S\}$, and

$$R = \{[qZq] \rightarrow i[qZq][qZq], [qZq] \rightarrow e\} \cup \{S \rightarrow [qZq]\}$$

If we replace $[qZq]$ by A we get the productions $S \rightarrow A$ and $A \rightarrow iAA|e$.

Example: Let $P = (\{p, q\}, \{0, 1\}, \{X, Z_0\}, \delta, q, Z_0)$, where δ is given by

$$1. \delta(q, 1, Z_0) = \{(q, XZ_0)\}$$

$$2. \delta(q, 1, X) = \{(q, XX)\}$$

$$3. \delta(q, 0, X) = \{(p, X)\}$$

$$4. \delta(q, \epsilon, X) = \{(q, \epsilon)\}$$

$$5. \delta(p, 1, X) = \{(p, \epsilon)\}$$

$$6. \delta(p, 0, Z_0) = \{(q, Z_0)\}$$

to a CFG.

We get $G = (V, \{0, 1\}, R, S)$, where

$V = \{[pXp], [pXq], [pZ_0p], [pZ_0q], [qXq], [qXp], [qZ_0q], [qZ_0p], S\}$ and the productions in R are

$$S \rightarrow [qZ_0q][qZ_0p]$$

From transition (1):

$$\begin{aligned}[qZ_0q] &\rightarrow 1[qXq][qZ_0q] \\ [qZ_0q] &\rightarrow 1[qXp][pZ_0q] \\ [qZ_0p] &\rightarrow 1[qXq][qZ_0p] \\ [qZ_0p] &\rightarrow 1[qXp][pZ_0p]\end{aligned}$$

From transition (2):

$$\begin{aligned}[qXq] &\rightarrow 1[qXq][qXq] \\ [qXq] &\rightarrow 1[qXp][pXq] \\ [qXp] &\rightarrow 1[qXq][qXp] \\ [qXp] &\rightarrow 1[qXp][pXp]\end{aligned}$$

From transition (3):

$$[qXq] \rightarrow 0[pXq]$$

$$[qXp] \rightarrow 0[pXp]$$

From transition (4):

$$[qXq] \rightarrow \epsilon$$

From transition (5):

$$[pXp] \rightarrow 1$$

From transition (6):

$$[pZ_0q] \rightarrow 0[qZ_0q]$$

$$[pZ_0p] \rightarrow 0[qZ_0p]$$

Theorem 6.14: Let G be constructed from a PDA P as above. Then $L(G) = N(P)$

Proof:

($\supseteq$ -direction.) We shall show by an induction on the length of the sequence $\vdash^*$ that

(♠) If $(q, w, X) \vdash^* (p, \epsilon, \epsilon)$ then $[qXp] \xRightarrow{*} w$.

Basis: Length 1. Then w is an a or ϵ , and $(p, \epsilon) \in \delta(q, w, X)$. By the construction of G we have $[qXp] \rightarrow w$ and thus $[qXp] \xRightarrow{*} w$.

Induction: Length is $n > 1$, and $\spadesuit$ holds for lengths $< n$. We must have a $\vdash$ sequence

$$(q, w, X) \vdash (r_0, x, Y_1 Y_2 \cdots Y_k) \vdash \cdots \vdash (p, \epsilon, \epsilon),$$

where $w = ax$ and $a \in \Sigma \cup \{\epsilon\}$. It follows that $(r_0, Y_1 Y_2 \cdots Y_k) \in \delta(q, a, X)$. Then we have a production

$$[qXr_k] \rightarrow a[r_0Y_1r_1] \cdots [r_{k-1}Y_kr_k],$$

for all $\{r_1, \dots, r_{k-1}\} \subseteq Q$ and $r_k = p$.

We may choose r_i to be the state of the PDA in the above $\vdash$ sequence when Y_i is popped. Let $w = aw_1w_2 \cdots w_k$, where w_i is consumed while Y_i is popped. Then

$$(r_{i-1}, w_i, Y_i) \vdash^* (r_i, \epsilon, \epsilon).$$

By the IH we get

$$[r_{i-1}Y_ir_i] \xRightarrow{*} w_i$$

We then get the following derivation sequence:

$$\begin{aligned}
[qXr_k] &\Rightarrow a[r_0Y_1r_1] \cdots [r_{k-1}Y_kr_k] \xRightarrow{*} \\
aw_1[r_1Y_2r_2][r_2Y_3r_3] \cdots [r_{k-1}Y_kr_k] &\xRightarrow{*} \\
aw_1w_2[r_2Y_3r_3] \cdots [r_{k-1}Y_kr_k] &\xRightarrow{*} \\
&\dots \\
aw_1w_2 \cdots w_k &= w
\end{aligned}$$

and $r_k = p$.

($\subseteq$ -direction.) We shall show by an induction on the length of the derivation $\xRightarrow{*}$ that

($\heartsuit$) If $[qXp] \xRightarrow{*} w$ then $(q, w, X) \vdash^* (p, \epsilon, \epsilon)$

Basis: One step. Then we have a production $[qXp] \rightarrow w$. From the construction of G it follows that $(p, \epsilon) \in \delta(q, a, X)$, where $w = a$. But then $(q, w, X) \vdash^* (p, \epsilon, \epsilon)$.

Induction: Length of $\xRightarrow{*}$ is $n > 1$, and $\heartsuit$ holds for lengths $< n$. Then we must have

$$[qXr_k] \Rightarrow a[r_0Y_1r_1][r_1Y_2r_2] \cdots [r_{k-1}Y_kr_k] \xRightarrow{*} w$$

where $r_k = p$. This must come from the transition $(r_0, Y_1Y_2 \cdots Y_k) \in \delta(q, a, X)$ and consequently

$$(q, aw_1w_2 \cdots w_k, X) \vdash (r_0, w_1w_2 \cdots w_k, Y_1Y_2 \cdots Y_k)$$

We can break w into $aw_1w_2\cdots w_k$ such that $[r_{i-1}Y_ir_i] \xRightarrow{*} w_i$. From the IH we get

$$(r_{i-1}, w_i, Y_i) \vdash^* (r_i, \epsilon, \epsilon)$$

From Theorem 6.5 we get

$$(r_{i-1}, w_iw_{i+1}\cdots w_k, Y_iY_{i+1}\cdots Y_k) \vdash^* (r_i, w_{i+1}\cdots w_k, Y_{i+1}\cdots Y_k)$$

Since this holds for all $i \in \{1, \dots, k\}$, we get

$$\begin{aligned} & (q, aw_1w_2\cdots w_k, X) \vdash \\ & (r_0, w_1w_2\cdots w_k, Y_1Y_2\cdots Y_k) \vdash^* \\ & (r_1, w_2\cdots w_k, Y_2\cdots Y_k) \vdash^* \\ & (r_2, w_3\cdots w_k, Y_3\cdots Y_k) \vdash^* \\ & \dots \\ & (p, \epsilon, \epsilon). \end{aligned}$$

So we have shown

$(q, w, X) \vdash^* (p, \epsilon, \epsilon)$ if and only if $[qXp] \xRightarrow{*} w$.

In particular for $q = q_0$ and $X = Z_0$

$(q_0, w, Z_0) \vdash^* (p, \epsilon, \epsilon)$ if and only if $[q_0Z_0p] \xRightarrow{*} w$

and because of the rule $S \rightarrow [q_0Z_0p]$

$[q_0Z_0p] \xRightarrow{*} w$ if and only if $S \xRightarrow{*} w$

Thus $w \in N(P)$ if and only if $w \in L(G)$

Deterministic PDA's

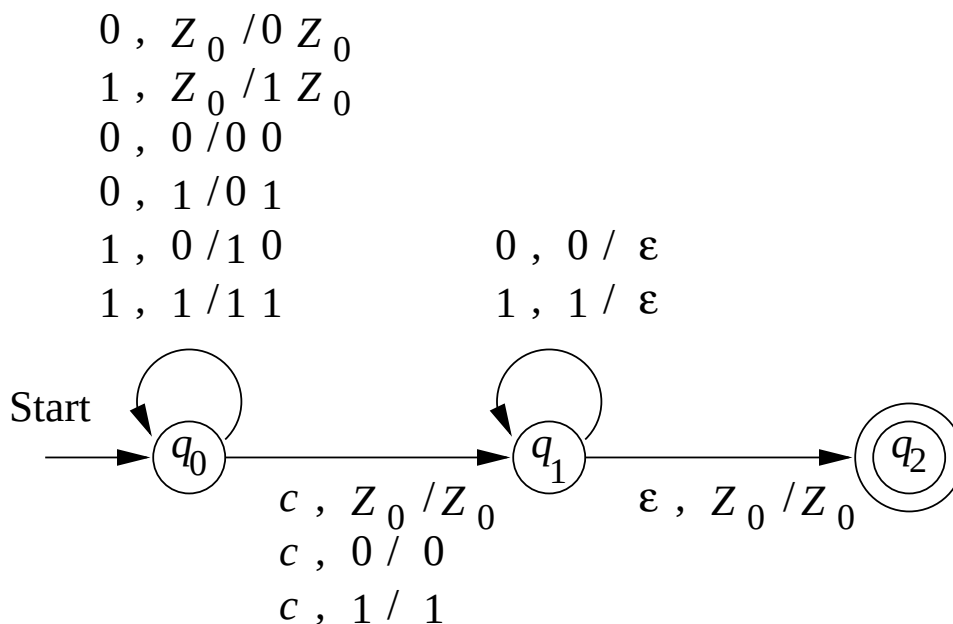
A PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ is *deterministic* iff

1. $\delta(q, a, X)$ and $\delta(q, \epsilon, X)$ are always empty or a singleton.
2. If $\delta(q, a, X)$ is nonempty, then $\delta(q, \epsilon, X)$ must be empty.

Example: Let us define

$$L_{wcwr} = \{wcw^R : w \in \{0, 1\}^*\}$$

Then L_{wcwr} is recognized by the following DPDA



We'll show that $\text{Regular} \subset L(\text{DPDA}) \subset \text{CFL}$

Theorem 6.17: If L is regular, then $L = L(P)$ for some DPDA P .

Proof: Since L is regular there is a DFA A s.t. $L = L(A)$. Let

$$A = (Q, \Sigma, \delta_A, q_0, F)$$

We define the DPDA

$$P = (Q, \Sigma, \{Z_0\}, \delta_P, q_0, Z_0, F),$$

where

$$\delta_P(q, a, Z_0) = \{(\delta_A(q, a), Z_0)\},$$

for all $p, q \in Q$, and $a \in \Sigma$.

An easy induction (do it!) on $|w|$ gives

$$(q_0, w, Z_0) \vdash^* (p, \epsilon, Z_0) \Leftrightarrow \hat{\delta}_A(q_0, w) = p$$

The theorem then follows (why?)

What about DPDA's that accept by null stack?

They cannot recognize all regular languages and only CFL's with the prefix property.

A language L has the *prefix property* if there are no two distinct strings in L , such that one is a prefix of the other.

Example: L_{wcr} has the prefix property.

Example: $\{0\}^*$ does not have the prefix property.

Theorem 6.19: L is $N(P)$ for some DPDA P if and only if L has the prefix property and L is $L(P')$ for some DPDA P' .

- We have seen that $\text{Regular} \subseteq L(\text{DPDA})$.
- $L_{wcwr} \in L(\text{DPDA}) \setminus \text{Regular}$
- $L_{wwr} \in \text{CFL} \setminus L(\text{DPDA})$

Therefore $\text{Regular} \subset L(\text{DPDA}) \subset \text{CFL}$.

- What about DPDA's and Ambiguous Grammars?

L_{wwr} has unamb. grammar $S \rightarrow 0S0 \mid 1S1 \mid \epsilon$
but is not $L(\text{DPDA})$.

For the converse we have

Theorem 6.20: If $L = N(P)$ for some DPDA P , then L has an unambiguous CFG.

Proof: By inspecting the proof of Theorem 6.14 we see that if the construction is applied to a DPDA the result is a CFG with unique leftmost derivations.

Theorem 6.20 can actually be strengthened as follows

Theorem 6.21: If $L = L(P)$ for some DPDA P , then L has an unambiguous CFG.

Proof: Let $\$$ be a symbol outside the alphabet of L , and let $L' = L\$$.

It is easy to see that L' has the prefix property. By Theorem 6.19 we have $L' = N(P')$ for some DPDA P' .

By Theorem 6.20 $N(P')$ can be generated by an unambiguous CFG G'

Modify G' into G , s.t. $L(G) = L$, by adding the production

$$\$ \rightarrow \epsilon$$

Since G' has unique leftmost derivations, G also has unique lm's, since the only new thing we're doing is adding derivations

$$w\$ \xRightarrow[lm]{} w$$

to the end.

Properties of CFL's

- *Simplification* of CFG's. This makes life easier, since we can claim that if a language is CF, then it has a grammar of a special form.
- *Pumping Lemma for CFL's*. Similar to the regular case.
- *Closure properties*. Some, but not all, of the closure properties of regular languages carry over to CFL's.
- *Decision properties*. We can test for membership and emptiness, but for instance, equivalence of CFL's is undecidable.

Chomsky Normal Form

We want to show that every CFL (without ϵ) is generated by a CFG where all productions are of the form

$$A \rightarrow BC, \text{ or } A \rightarrow a$$

where A, B , and C are variables, and a is a terminal. This is called CNF, and to get there we have to first 'clean up' the grammar:

1. Eliminate *ϵ -productions*, that is, productions of the form $A \rightarrow \epsilon$.
2. Eliminate *unit productions*, that is, productions of the form $A \rightarrow B$, where A and B are variables.
3. Eliminate *useless symbols*, those that do not appear in any derivation $S \xRightarrow{*} w$, for start symbol S and terminal string w .

Eliminating Useless Symbols

- A symbol $X \in V \cup T$ is *useful* for a grammar $G = (V, T, P, S)$, if there is a derivation

$$S \xRightarrow[G]{*} \alpha X \beta \xRightarrow[G]{*} w$$

for a terminal string w . Symbols that are not useful are called *useless*.

- A symbol X is *generating* if $X \xRightarrow[G]{*} w$, for some $w \in T^*$. Note that every terminal t is generating as $t \xRightarrow[G]{*} t$.

- A symbol X is *reachable* if $S \xRightarrow[G]{*} \alpha X \beta$, for some $\{\alpha, \beta\} \subseteq (V \cup T)^*$

It turns out that if we eliminate non-generating symbols first, and then non-reachable ones, we will be left with only useful symbols.

Example: Let G be

$$S \rightarrow AB|a, A \rightarrow b$$

S and A are generating, B is not. If we eliminate B we have to eliminate $S \rightarrow AB$, leaving the grammar

$$S \rightarrow a, A \rightarrow b$$

Now only S and a are reachable. Eliminating A and b leaves us with

$$S \rightarrow a$$

with language $\{a\}$.

On the other hand, if we eliminate non-reachable symbols first, we find that all symbols are reachable. From

$$S \rightarrow AB|a, A \rightarrow b$$

we then eliminate B as non-generating, and are left with

$$S \rightarrow a, A \rightarrow b$$

that still contains useless symbols

Theorem 7.2: Let $G = (V, T, P, S)$ be a CFG such that $L(G) \neq \emptyset$. Let $G_1 = (V_1, T_1, P_1, S)$ be the grammar obtained by

1. Eliminating all nongenerating symbols and the productions they occur in. Let the new grammar be $G_2 = (V_2, T_2, P_2, S)$.
2. Eliminate from G_2 all nonreachable symbols and the productions they occur in.

Then G_1 has no useless symbols, and $L(G_1) = L(G)$.

Proof: We first prove that G_1 has no useless symbols:

Let X remain in $V_1 \cup T_1$.

Since X was not eliminated in step 1, $X \xRightarrow{*} w$ in G , for some $w \in T^*$. Moreover, every symbol used in this derivation is also generating. Thus $X \xRightarrow{*} w$ in G_2 also.

Since X was not eliminated in step 2, there are α and β , such that $S \xRightarrow{*} \alpha X \beta$ in G_2 . Furthermore, every symbol used in this derivation is also reachable, so $S \xRightarrow{*} \alpha X \beta$ in G_1 .

Now every symbol in $\alpha X \beta$ is reachable and in $V_2 \cup T_2 \supseteq V_1 \cup T_1$, so each of them is generating in G_2 .

The terminal derivation $\alpha X \beta \xRightarrow{*} xwy$ in G_2 involves only symbols that are reachable from S , because they are reached by symbols in $\alpha X \beta$.

Thus the terminal derivation is also a derivation of G_1 , i.e.,

$$S \xRightarrow{*} \alpha X \beta \xRightarrow{*} xwy$$

in G_1 . We conclude that X is useful in G_1 .

We then show that $L(G_1) = L(G)$.

Since $P_1 \subseteq P$, we have $L(G_1) \subseteq L(G)$.

Then, let $w \in L(G)$. Thus $S \xRightarrow[G]{*} w$. Each symbol in this derivation is evidently both reachable and generating, so this is also a derivation of G_1 .

Thus $w \in L(G_1)$.

We have to give algorithms to compute the generating and reachable symbols of $G = (V, T, P, S)$.

The generating symbols $g(G)$ are computed by the following closure algorithm:

Basis: $g(G) = T$

Induction: If for all $\alpha_i \in \alpha$, $\alpha_i \in g(G)$ and $X \rightarrow \alpha \in P$, then $g(G) = g(G) \cup \{X\}$.

Example: Let G be $S \rightarrow AB|a$, $A \rightarrow b$

Then first $g(G) = \{a, b\}$.

Since $S \rightarrow a$ we put S in $g(G)$, and because $A \rightarrow b$ we add A also, and that's it.

Theorem 7.4: At saturation, $g(G)$ contains all and only the generating symbols of G .

Proof:

We'll show in class on an induction on the stage in which a symbol X is added to $g(G)$ that X is indeed generating.

Then, suppose that X is generating. Thus $X \xRightarrow[G]{*} w$, for some $w \in T^*$. We prove by induction on this derivation that $X \in g(G)$.

Basis: Zero Steps. Then X is added in the basis of the closure algo.

Induction: The derivation takes $n > 0$ steps. Let the first production used be $X \rightarrow \alpha$. Then

$$X \Rightarrow \alpha \xRightarrow{*} w$$

and $\alpha \xRightarrow{*} w$ in less than n steps and by the IH $\alpha \in g(G)$. From the inductive part of the algo it follows that $X \in g(G)$.

The set of reachable symbols $r(G)$ of $G = (V, T, P, S)$ is computed by the following closure algorithm:

Basis: $r(G) = \{S\}$.

Induction: If variable $A \in r(G)$ and $A \rightarrow \alpha \in P$ then add all symbols in α to $r(G)$

Example: Let G be $S \rightarrow AB|a, A \rightarrow b$

Then first $r(G) = \{S\}$.

Based on the first production we add $\{A, B, a\}$ to $r(G)$.

Based on the second production we add $\{b\}$ to $r(G)$ and that's it.

Theorem 7.6: At saturation, $r(G)$ contains all and only the reachable symbols of G .

Eliminating ϵ -Productions

We shall prove that if L is CF, then $L \setminus \{\epsilon\}$ has a grammar without ϵ -productions.

Variable A is said to be *nullable* if $A \xRightarrow{*} \epsilon$.

Let A be nullable. We'll then replace a rule like

$$A \rightarrow BAD$$

with

$$A \rightarrow BAD, A \rightarrow BD$$

and delete any rules with body ϵ .

We'll compute $n(G)$, the set of nullable symbols of a grammar $G = (V, T, P, S)$ as follows:

Basis: $n(G) = \{A : A \rightarrow \epsilon \in P\}$

Induction: If $\{C_1, C_2, \dots, C_k\} \subseteq n(G)$ and $A \rightarrow C_1 C_2 \dots C_k \in P$, then $n(G) = n(G) \cup \{A\}$.

Theorem 7.7: At saturation, $n(G)$ contains all and only the nullable symbols of G .

Proof: Easy induction in both directions.

Once we know the nullable symbols, we can transform G into G_1 as follows:

- For each $A \rightarrow X_1X_2 \cdots X_k \in P$ with $m \leq k$ nullable symbols, replace it by 2^m rules, one with each sublist of the nullable symbols absent.

Exception: If $m = k$ we don't delete all m nullable symbols.

- Delete all rules of the form $A \rightarrow \epsilon$.

Example: Let G be

$$S \rightarrow AB, A \rightarrow aAA|\epsilon, B \rightarrow bBB|\epsilon$$

Now $n(G) = \{A, B, S\}$. The first rule will become

$$S \rightarrow AB|A|B$$

the second

$$A \rightarrow aAA|aA|aA|a$$

the third

$$B \rightarrow bBB|bB|bB|b$$

We then delete rules with ϵ -bodies, and end up with grammar G_1 :

$$S \rightarrow AB|A|B, A \rightarrow aAA|aA|a, B \rightarrow bBB|bB|b$$

Theorem 7.9: $L(G_1) = L(G) \setminus \{\epsilon\}$.

Proof: We'll prove the stronger statement:

$A \xRightarrow{*} w$ in G_1 if and only if $w \neq \epsilon$ and $A \xRightarrow{*} w$ in G .

The theorem then follows by choosing $A = S$.

$\subseteq$ -direction: Suppose $A \xRightarrow{*} w$ in G_1 . Then clearly $w \neq \epsilon$ (Why?). We'll show by induction on the length of the derivation that $A \xRightarrow{*} w$ in G also.

Basis: One step. Then there exists $A \rightarrow w$ in G_1 . From the construction of G_1 it follows that there exists $A \rightarrow \alpha$ in G , where α is w plus some nullable variables interspersed. Then

$$A \Rightarrow \alpha \xRightarrow{*} w$$

in G .

Induction: Derivation takes $n > 1$ steps. Then

$$A \Rightarrow X_1 X_2 \cdots X_k \xRightarrow{*} w \text{ in } G_1$$

and the first derivation is based on a production

$$A \rightarrow Y_1 Y_2 \cdots Y_m \text{ in } G$$

where $m \geq k$, some Y_i 's are X_j 's and the other are nullable symbols of G .

Furhtermore, $w = w_1 w_2 \cdots w_k$, and $X_i \xRightarrow{*} w_i$ in G_1 in less than n steps. By the IH we have $X_i \xRightarrow{*} w_i$ in G . Now we get

$$A \xRightarrow{G} Y_1 Y_2 \cdots Y_m \xRightarrow{*G} X_1 X_2 \cdots X_k \xRightarrow{*G} w_1 w_2 \cdots w_k = w$$

$\supseteq$ -direction: Let $A \xRightarrow[G]{*} w$, and $w \neq \epsilon$. We'll show by induction of the length of the derivation that $A \xRightarrow{*} w$ in G_1 .

Basis: Length is one. Then $A \rightarrow w$ is in G , and since $w \neq \epsilon$ the rule is in G_1 also.

Induction: Derivation takes $n > 1$ steps. Then it looks like

$$A \xRightarrow[G]{*} Y_1 Y_2 \cdots Y_m \xRightarrow[G]{*} w$$

Now $w = w_1 w_2 \cdots w_m$, and $Y_i \xRightarrow[G]{*} w_i$ in less than n steps.

Let $X_1 X_2 \cdots X_k$ be those Y_j 's in order, such that $w_j \neq \epsilon$. Then $A \rightarrow X_1 X_2 \cdots X_k$ is a rule in G_1 .

Now $X_1 X_2 \cdots X_k \xRightarrow[G]{*} w$ (Why?)

Each $X_j/Y_j \xRightarrow[G]{*} w_j$ in less than n steps, so by IH we have that if $w \neq \epsilon$ then $Y_j \xRightarrow{*} w_j$ in G_1 . Thus

$$A \Rightarrow X_1 X_2 \cdots X_k \xRightarrow{*} w \text{ in } G_1$$

Eliminating Unit Productions

$$A \rightarrow B$$

is a *unit* production, whenever A and B are variables.

Unit productions can be eliminated.

Let's look at grammar

$$I \rightarrow a \mid b \mid Ia \mid Ib \mid IO \mid I1$$

$$F \rightarrow I \mid (E)$$

$$T \rightarrow F \mid T * F$$

$$E \rightarrow T \mid E + T$$

It has unit productions $E \rightarrow T$, $T \rightarrow F$, and $F \rightarrow I$

We'll expand rule $E \rightarrow T$ and get rules

$$E \rightarrow F, E \rightarrow T * F$$

We then expand $E \rightarrow F$ and get

$$E \rightarrow I|(E)|T * F$$

Finally we expand $E \rightarrow I$ and get

$$E \rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \mid (E) \mid T * F$$

The expansion method works as long as there are no cycles in the rules, as e.g. in

$$A \rightarrow B, B \rightarrow C, C \rightarrow A$$

The following method based on *unit pairs* will work for all grammars.

(A, B) is a *unit pair* if $A \xRightarrow{*} B$ using unit productions only.

Note: In $A \rightarrow BC, C \rightarrow \epsilon$ we have $A \xRightarrow{*} B$, but not using unit productions only.

To compute $u(G)$, the set of all unit pairs of $G = (V, T, P, S)$ we use the following closure algorithm

Basis: $u(G) = \{(A, A) : A \in V\}$

Induction: If $(A, B) \in u(G)$ and $B \rightarrow C \in P$ then add (A, C) to $u(G)$.

Theorem 7.11: At saturation, $u(G)$ contains all and only the unit pairs of G .

Given $G = (V, T, P, S)$ we can construct

$G_1 = (V, T, P_1, S)$ without unit productions,
and such that $L(G_1) = L(G)$ by setting:

$$P_1 = \{A \rightarrow \alpha : \alpha \notin V, B \rightarrow \alpha \in P, (A, B) \in u(G)\}$$

Example: From the expression grammar

$$I \rightarrow a \mid b \mid Ia \mid Ib \mid IO \mid I1$$

$$F \rightarrow I \mid (E)$$

$$T \rightarrow F \mid T * F$$

$$E \rightarrow T \mid E + T$$

we get:

Pair	Productions
(E, E)	$E \rightarrow E + T$
(E, T)	$E \rightarrow T * F$
(E, F)	$E \rightarrow (E)$
(E, I)	$E \rightarrow a \mid b \mid Ia \mid Ib \mid IO \mid I1$
(T, T)	$T \rightarrow T * F$
(T, F)	$T \rightarrow (E)$
(T, I)	$T \rightarrow a \mid b \mid Ia \mid Ib \mid IO \mid I1$
(F, F)	$F \rightarrow (E)$
(F, I)	$F \rightarrow a \mid b \mid Ia \mid Ib \mid IO \mid I1$
(I, I)	$I \rightarrow a \mid b \mid Ia \mid Ib \mid IO \mid I1$

The resulting grammar is equivalent to the original one.

Summary

To “clean up” a grammar we can

1. Eliminate ϵ -productions
2. Eliminate unit productions
3. Eliminate useless symbols

in this order.

Chomsky Normal Form, CNF

We shall show that every nonempty CFL without ϵ has a grammar G without useless symbols, and such that every production is of the form

- $A \rightarrow BC$, where $\{A, B, C\} \subseteq V$, or
- $A \rightarrow \alpha$, where $A \in V$, and $\alpha \in T$.

To achieve this, start with any grammar for the CFL, and

1. “Clean up” the grammar.
2. Arrange that all bodies of length 2 or more consist of only variables.
3. Break bodies of length 3 or more into a cascade of two-variable-bodied productions.

- For step 2, for every terminal a that appears in a body of length ≥ 2 , create a new variable, say A , and replace a by A in all bodies.

Then add a new rule $A \rightarrow a$.

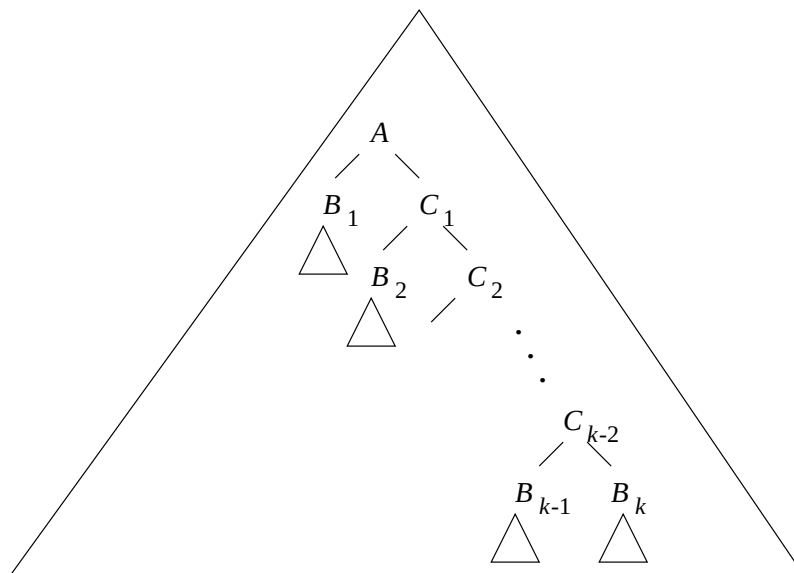
- For step 3, for each rule of the form

$$A \rightarrow B_1 B_2 \cdots B_k,$$

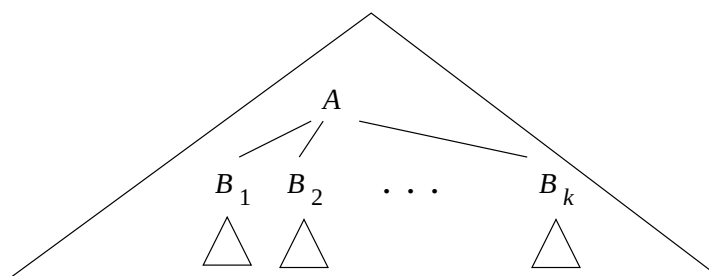
$k \geq 3$, introduce new variables $C_1, C_2, \dots, C_{k-2}$, and replace the rule with

$$\begin{aligned} A &\rightarrow B_1 C_1 \\ C_1 &\rightarrow B_2 C_2 \\ &\dots \\ C_{k-3} &\rightarrow B_{k-2} C_{k-2} \\ C_{k-2} &\rightarrow B_{k-1} B_k \end{aligned}$$

Illustration of the effect of step 3



(a)



(b)

Example of CNF conversion

Let's start with the grammar (step 1 already done)

$$\begin{aligned}E &\rightarrow E + T \mid T * F \mid (E) \mid a \mid b \mid Ia \mid Ib \mid IO \mid I1 \\T &\rightarrow T * F \mid (E) \mid a \mid b \mid Ia \mid Ib \mid IO \mid I1 \\F &\rightarrow (E) \mid a \mid b \mid Ia \mid Ib \mid IO \mid I1 \\I &\rightarrow a \mid b \mid Ia \mid Ib \mid IO \mid I1\end{aligned}$$

For step 2, we need the rules

$$A \rightarrow a, B \rightarrow b, Z \rightarrow 0, O \rightarrow 1$$

$$P \rightarrow +, M \rightarrow *, L \rightarrow (, R \rightarrow)$$

and by replacing we get the grammar

$$\begin{aligned}E &\rightarrow EPT \mid TMF \mid LER \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\T &\rightarrow TMF \mid LER \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\F &\rightarrow LER \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\I &\rightarrow a \mid b \mid IA \mid IB \mid IZ \mid IO \\A &\rightarrow a, B \rightarrow b, Z \rightarrow 0, O \rightarrow 1 \\P &\rightarrow +, M \rightarrow *, L \rightarrow (, R \rightarrow)\end{aligned}$$

For step 3, we replace

$$E \rightarrow EPT \text{ by } E \rightarrow EC_1, C_1 \rightarrow PT$$

$$\begin{aligned} E \rightarrow TMF, T \rightarrow TMF \text{ by} \\ E \rightarrow TC_2, T \rightarrow TC_2, C_2 \rightarrow MF \end{aligned}$$

$$\begin{aligned} E \rightarrow LER, T \rightarrow LER, F \rightarrow LER \text{ by} \\ E \rightarrow LC_3, T \rightarrow LC_3, F \rightarrow LC_3, C_3 \rightarrow ER \end{aligned}$$

The final CNF grammar is

$$\begin{aligned} E &\rightarrow EC_1 \mid TC_2 \mid LC_3 \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\ T &\rightarrow TC_2 \mid LC_3 \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\ F &\rightarrow LC_3 \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\ I &\rightarrow a \mid b \mid IA \mid IB \mid IZ \mid IO \\ C_1 &\rightarrow PT, C_2 \rightarrow MF, C_3 \rightarrow ER \\ A &\rightarrow a, B \rightarrow b, Z \rightarrow 0, O \rightarrow 1 \\ P &\rightarrow +, M \rightarrow *, L \rightarrow (, R \rightarrow) \end{aligned}$$

Closure Properties of CFL's

Consider a mapping from symbols to languages

$$s : \Sigma \rightarrow 2^{(\Delta^*)}$$

where Σ and Δ are finite alphabets. Let $w \in \Sigma^*$, where $w = a_1a_2 \cdots a_n$, and define

$$s(a_1a_2 \cdots a_n) = s(a_1).s(a_2).\cdots.s(a_n)$$

and, for $L \subseteq \Sigma^*$,

$$s(L) = \bigcup_{w \in L} s(w)$$

Such a mapping s is called a *substitution*.

Example: $\Sigma = \{0, 1\}$, $\Delta = \{a, b\}$,
 $s(0) = \{a^n b^n : n \geq 1\}$, $s(1) = \{aa, bb\}$.

Let $w = 01$. Then $s(w) = s(0).s(1) =$
 $\{a^n b^n aa : n \geq 1\} \cup \{a^n b^{n+2} : n \geq 1\}$

Let $L = \{0\}^*$. Then $s(L) = (s(0))^* =$
 $\{a^{n_1} b^{n_1} a^{n_2} b^{n_2} \dots a^{n_k} b^{n_k} : k \geq 0, n_i \geq 1\}$

Theorem 7.23: Let L be a CFL over Σ , and s a substitution, such that $s(a)$ is a CFL, $\forall a \in \Sigma$. Then $s(L)$ is a CFL.

We start with grammars

$$G = (V, \Sigma, P, S)$$

for L , and

$$G_a = (V_a, T_a, P_a, S_a)$$

for each $s(a)$. We then construct

$$G' = (V', T', P', S)$$

for $s(L)$, where

$$V' = (\cup_{a \in \Sigma} V_a) \cup V$$

The sets of variables must be disjoint! Rename if necessary.

$$T' = \cup_{a \in \Sigma} T_a$$

$P' = \cup_{a \in \Sigma} P_a$ plus the productions of P with each a in a body replaced with symbol S_a .

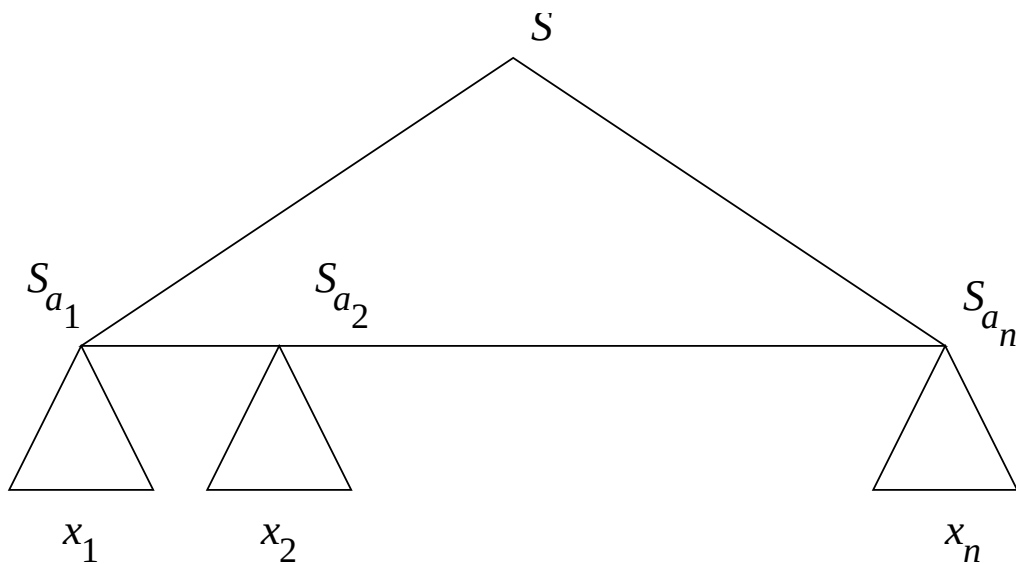
Now we have to show that

- $L(G') = s(L) = \bigcup_{x \in L} s(x).$

Let $w \in s(L)$. Then $\exists x = a_1 a_2 \cdots a_n$ in L , such that $w = s(x)$ and $\exists x_i \in s(a_i)$, such that $w = x_1 x_2 \cdots x_n$.

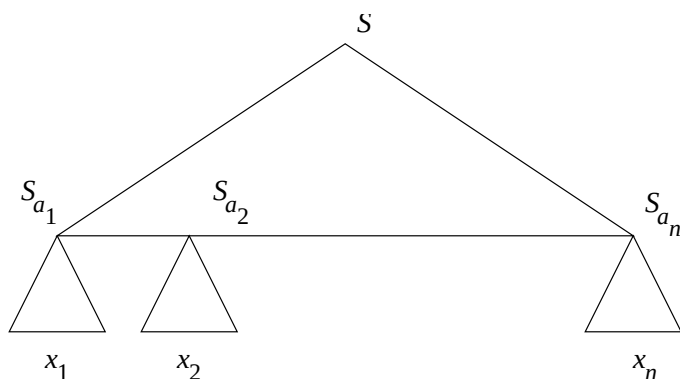
Thus $x \in L(G)$ and $x_i \in L(G_{a_i})$.

A parse tree in G' will look like



Thus we can generate $s_{a_1} s_{a_2} \cdots s_{a_n}$ in G' and from there we generate $x_1 x_2 \cdots x_n = w$. Thus $w \in L(G')$.

Let $w \in L(G')$. Then the parse tree for w must again look like



Indeed, as the sets of variables V and V_a are all disjoint, the upper part of the parse tree must use only productions of G until reaching S_a . Below S_a , only productions of G_a may be used.

For each dangling subtree, the yield is $x_i \in L(G_{a_i}) = s(a_i)$. So $w = x_1 \cdots x_n \in s(a_1 \cdots a_n)$. Now delete the dangling subtrees. Then the yield is

$$S_{a_1} S_{a_2} \cdots S_{a_n}$$

where $a_1 a_2 \cdots a_n \in L(G) = L$. Thus w is in $s(L)$.

Applications of the Substitution Theorem

Theorem 7.24: The CFL's are closed under (i) : union, (ii) : concatenation, (iii) : Kleene closure and positive closure $+$, and (iv) : homomorphism.

Proof: (i): Let L_1 and L_2 be CFL's, let $L = \{1, 2\}$, and $s(1) = L_1, s(2) = L_2$.

Then $L_1 \cup L_2 = s(L)$.

(ii) : Here we choose $L = \{12\}$ and s as before. Then $L_1.L_2 = s(L)$

(iii) : Suppose L_1 is CF. Let $L = \{1\}^*, s(1) = L_1$. Now $L_1^* = s(L)$. Similar proof for $+$.

(iv) : Let L be a CFL over Σ , and h a homomorphism on Σ . Then define s by

$$a \mapsto \{h(a)\}$$

Then $h(L) = s(L)$.

Theorem 7.25: If L is CF, then so is L^R .

Proof: Suppose L is generated by $G = (V, T, P, S)$. Construct $G^R = (V, T, P^R, S)$, where

$$P^R = \{A \rightarrow \alpha^R : A \rightarrow \alpha \in P\}$$

All the sentential forms of G^R are reverses of sentential forms of G and vice-versa.

We can show by inductions on the lengths of the derivations in G (for one direction) and in G^R (for the other direction) that $(L(G))^R = L(G^R) = L^R$.

Example: CFL's are not closed under intersection.

Let $L_1 = \{0^n 1^n 2^i : n \geq 1, i \geq 1\}$. Then L_1 is CF with grammar

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow 0A1 \mid 01 \\ B &\rightarrow 2B \mid 2 \end{aligned}$$

Also, $L_2 = \{0^i 1^n 2^n : n \geq 1, i \geq 1\}$ is CF with grammar

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow 0A \mid 0 \\ B &\rightarrow 1B2 \mid 12 \end{aligned}$$

However, $L_1 \cap L_2 = \{0^n 1^n 2^n : n \geq 1\}$ which is not CF (see Example 7.19 in your book).

Theorem 7.27: If L is CF, and R regular, then $L \cap R$ is CF.

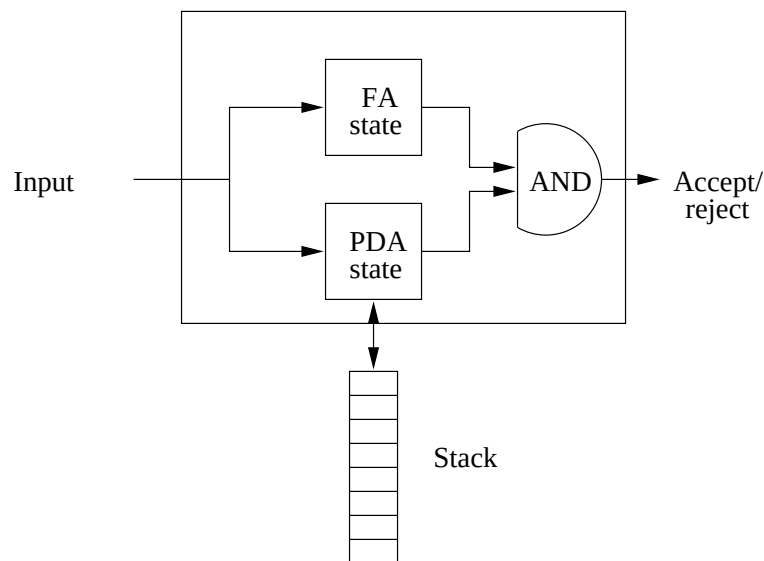
Proof: Let L be accepted by PDA

$$P = (Q_P, \Sigma, \Gamma, \delta_P, q_P, Z_0, F_P)$$

by final state, and let R be accepted by DFA

$$A = (Q_A, \Sigma, \delta_A, q_A, F_A)$$

We'll construct a PDA for $L \cap R$ according to the picture



Formally, define

$$P' = (Q_P \times Q_A, \Sigma, \Gamma, \delta, (q_P, q_A), Z_0, F_P \times F_A)$$

where

$$\delta((q, p), a, X) = \{((r, \hat{\delta}_A(p, a)), \gamma) : (r, \gamma) \in \delta_P(q, a, X)\}$$

Note that a may be a symbol of Σ or $a = \epsilon$.

We can prove by an induction on the number of moves ($\vdash^*$) by the PDA's P and P' that

$$(q_P, w, Z_0) \vdash^* (q, \epsilon, \gamma) \text{ in } P$$

if and only if

$$((q_P, q_A), w, Z_0) \vdash^* ((q, \hat{\delta}(q_A, w)), \epsilon, \gamma) \text{ in } P'$$

The claim then follows (Why?)

Theorem 7.29: Let L, L_1, L_2 be CFL's and R regular. Then

1. $L \setminus R$ is CF
2. $\bar{L}$ is not necessarily CF
3. $L_1 \setminus L_2$ is not necessarily CF

Proof:

1. $\bar{R}$ is regular, $L \cap \bar{R}$ is context-free, and $L \cap \bar{R} = L \setminus R$.
2. If $\bar{L}$ always was CF, it would follow that

$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$$

always would be CF.

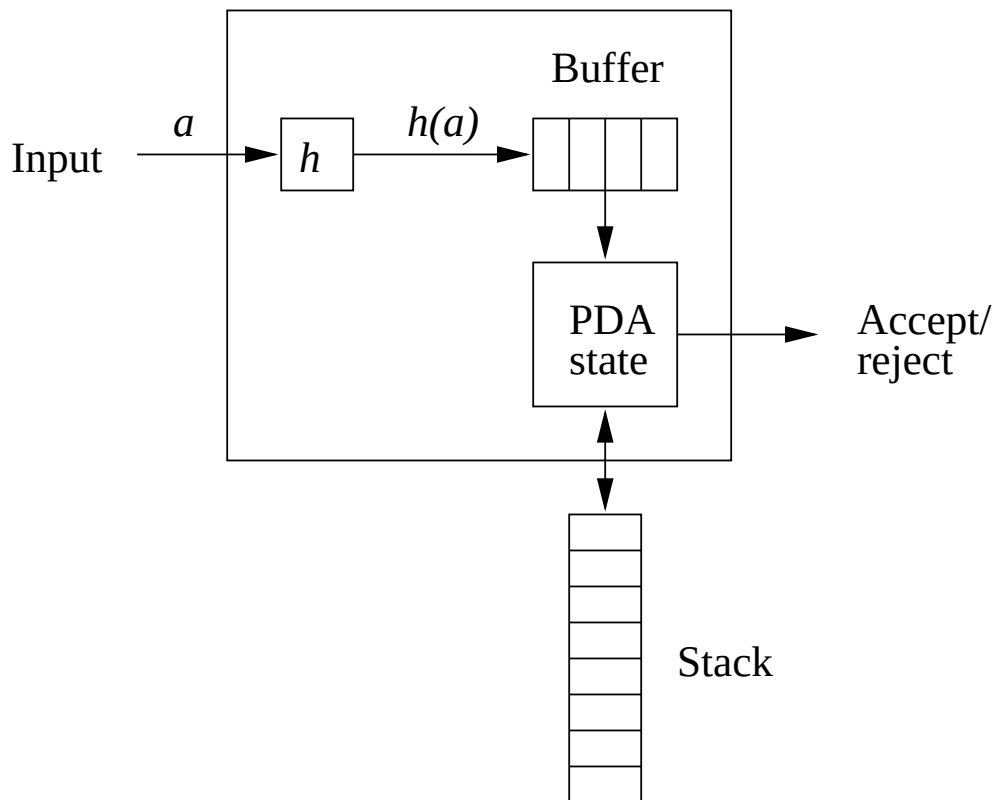
3. Note that Σ^* is CF, so if $L_1 \setminus L_2$ was always CF, then so would $\Sigma^* \setminus L = \bar{L}$.

Inverse homomorphism

Let $h : \Sigma \rightarrow \Theta^*$ be a homomorphism and $L \subseteq \Theta^*$. We define the inverse homomorphism as $h^{-1}(L) = \{w \in \Sigma^* : h(w) \in L\}$

Theorem 7.30: Let L be a CFL, and h a homomorphism. Then $h^{-1}(L)$ is a CFL.

Proof: The plan of the proof is



Let L be accepted by PDA

$$P = (Q, \Theta, \Gamma, \delta, q_0, Z_0, F)$$

We construct a new PDA

$$P' = (Q', \Sigma, \Gamma, \delta', (q_0, \epsilon), Z_0, F \times \{\epsilon\})$$

where

$$Q' = \{(q, x) : q \in Q, x \in \text{suffix}(h(a)), a \in \Sigma\}$$

$$\delta'((q, \epsilon), a, X) = \{((q, h(a)), X) : a \in \Sigma, \\ q \in Q, X \in \Gamma\} \text{ Note that } a \neq \epsilon.$$

$$\delta'((q, bx), \epsilon, X) = \{((p, x), \gamma) : (p, \gamma) \in \delta(q, b, X), \\ b \in \Theta \cup \{\epsilon\}, q \in Q, X \in \Gamma\}$$

We can show by suitable inductions on the number of moves made by the automata that

- $(q_0, h(w), Z_0) \vdash^* (p, \epsilon, \gamma)$ in P if and only if $((q_0, \epsilon), w, Z_0) \vdash^* ((p, \epsilon), \epsilon, \gamma)$ in P' .

Decision Properties of CFL's

We'll look at the following:

- Complexity of converting among CFG's and PDA's
- Converting a CFG to CNF
- Testing $L(G) \neq \emptyset$, for a given G
- Testing $w \in L(G)$, for a given w and fixed G .
- Preview of undecidable CFL problems

Converting between CFA's and PDA's

- Input size is n .
- n is the *total* size of the input CFG or PDA.

The following work in time $O(n)$

1. Converting a CFG to a PDA
2. Converting a “final state” PDA to a “null stack” PDA
3. Converting a “null stack” PDA to a “final state” PDA

Avoidable exponential blow-up

For converting a PDA to a CFG we have:

At most n^3 variables of the form $[pXq]$

If $(r, Y_1Y_2 \cdots Y_k) \in \delta(q, a, X)\}$, we'll have $O(n^n)$ rules of the form

$$[qXr_k] \rightarrow a[rY_1r_1] \cdots [r_{k-1}Y_kr_k]$$

- By introducing $k-2$ new states we can modify the PDA to push at most *one* symbol per transition.

- Now, k will be ≤ 2 for all rules.
- Total length of all transitions is still $O(n)$.
- Now, each transition generates at most n^2 productions
- Total size (and time to calculate) the grammar is therefore $O(n^3)$.

Converting into CNF

Good news:

1. Computing $r(G)$ and $g(G)$ and eliminating useless symbols takes time $O(n)$. This will be shown shortly
2. Size of $u(G)$ and the resulting grammar with productions P_1 is $O(n^2)$
3. Arranging that bodies consist of only variables is $O(n)$
4. Breaking of bodies is $O(n)$

Bad news:

- Eliminating the nullable symbols can make the new grammar have size $O(2^n)$

The bad news is avoidable:

Break bodies first before eliminating nullable symbols

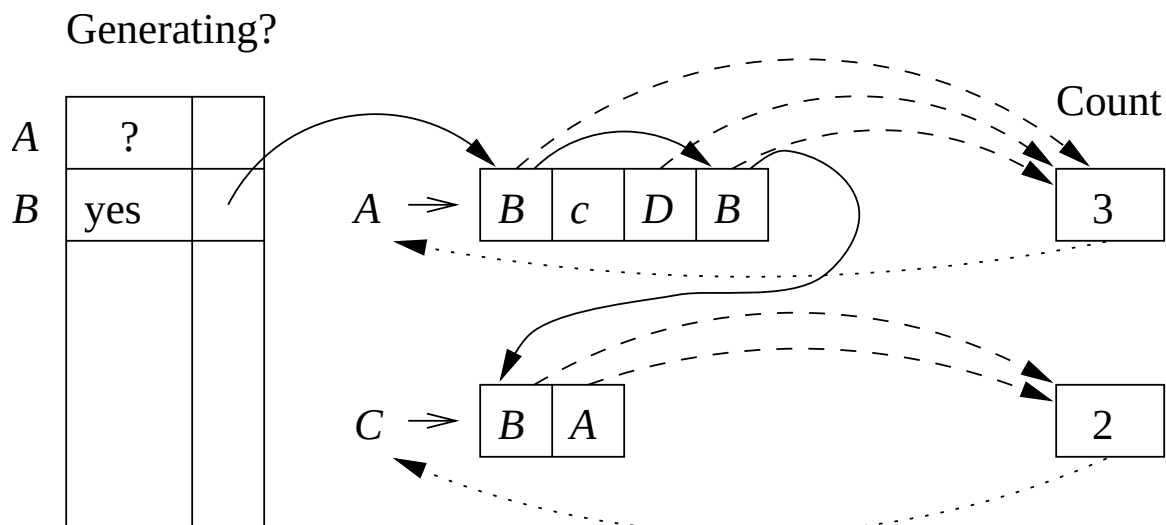
- Conversion into CNF is $O(n^2)$

Testing emptiness of CFL's

$L(G)$ is non-empty if the start symbol S is generating.

A naive implementation on $g(G)$ takes time $O(n^2)$.

$g(G)$ can be computed in time $O(n)$ as follows:



Creation and initialization of the array is $O(n)$.

Creation and initialization of the links and counts is $O(n)$.

When a count goes to zero, we have to

1. Finding the head variable A , checking if it already is “yes” in the array, and if not, queueing it is $O(1)$ per production. Total $O(n)$.
2. Following links for A , and decreasing the counters. Takes time $O(n)$.

Total time is $O(n)$.

$$w \in L(G)?$$

Inefficient way:

Suppose G is CNF, test string is w , with $|w| = n$. Since the parse tree is binary, there are $2n - 1$ internal nodes.

Generate *all* binary parse trees of G with $2n - 1$ internal nodes.

Check if any parse tree generates w .

CYK-algo for membership testing

The CNF grammar G is fixed

Input is $w = a_1 a_2 \cdots a_n$

We construct a triangular table, where X_{ij} contains all variables A , such that

$$A \xRightarrow[G]{*} a_i a_{i+1} \cdots a_j$$

X_{15}				
X_{14}	X_{25}			
X_{13}	X_{24}	X_{35}		
X_{12}	X_{23}	X_{34}	X_{45}	
X_{11}	X_{22}	X_{33}	X_{44}	X_{55}
a_1	a_2	a_3	a_4	a_5

To fill the table we work row-by-row, upwards.

Basis (first row): $X_{ii} = \{A : A \rightarrow a_i \text{ is in } G\}$

Induction (other rows):

We wish to compute X_{ij} , which is in row $j - i + 1$.
We know that $j > i$.

$A \in X_{ij}$, i.e. $A \xRightarrow{*} a_i a_{i+1} \cdots a_j$, if we find variables B and C and integer k such that:

1. $A \rightarrow BC$,

2. $i \leq k < j$,

3. $B \in X_{ik}$, i.e. $B \xRightarrow{*} a_i a_{i+1} \cdots a_k$, and

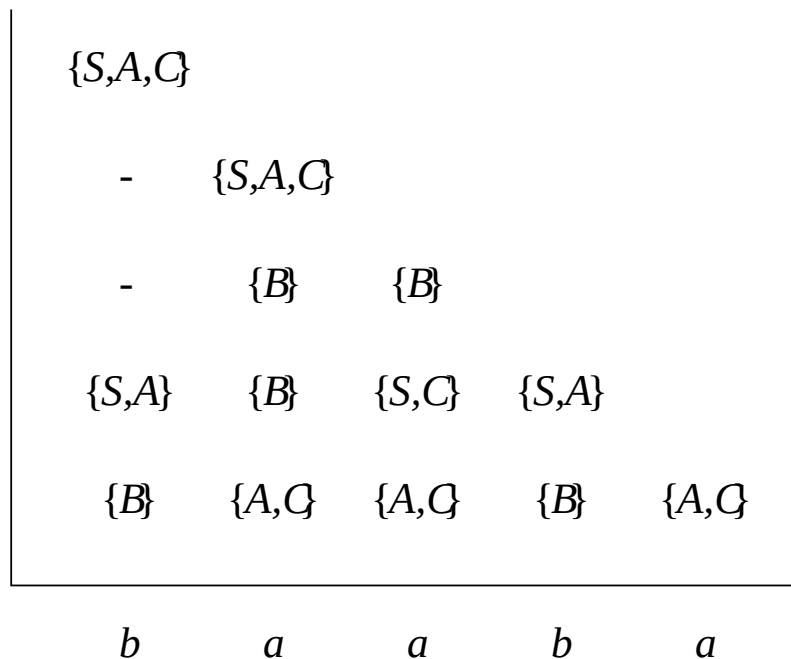
4. $C \in X_{k+1j}$, i.e. $C \xRightarrow{*} a_{k+1} a_{k+2} \cdots a_j$.

Theorem 7.33: The algorithm described above correctly computes X_{ij} for all i and j . Thus $w \in L(G)$ if and only if $S \in X_{1n}$.

Example:

G has productions

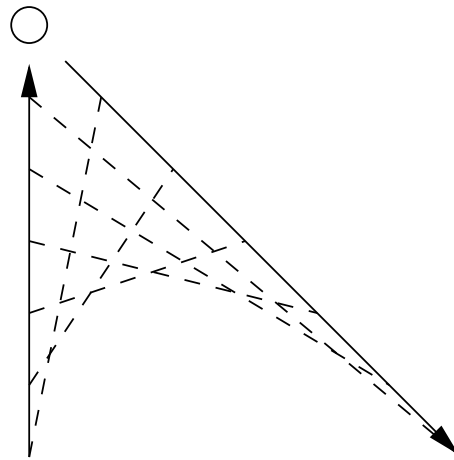
$$\begin{aligned} S &\rightarrow AB|BC \\ A &\rightarrow BA|a \\ B &\rightarrow CC|b \\ C &\rightarrow AB|a \end{aligned}$$



To compute X_{ij} we need to compare at most n pairs of previously computed sets:

$$(X_{ii}, X_{i+1,j}), (X_{i,i+1}, X_{i+2,j}), \dots, (X_{i,j-1}, X_{jj})$$

as suggested below



For $w = a_1 \cdots a_n$, there are $O(n^2)$ entries X_{ij} to compute.

For each X_{ij} we need to compare at most n pairs $(X_{ik}, X_{k+1,j})$.

Total work is $O(n^3)$.

Preview of undecidable CFL problems

The following are undecidable (= have no algorithm):

1. Is a given CFG G ambiguous? (Theorem 9.20)
2. Is a given CFL inherently ambiguous?
3. Is the intersection of two CFL's empty? (Theorem 9.22)
4. Are two CFL's the same? (Theorem 9.22)
5. Is a given CFL universal (equal to Σ^*)? (Theorem 9.22)

Problems that computers cannot solve

Evidently, it is important to know that programs do what they are supposed to, i.e. we would like to make sure that programs are *correct*.

It is easy to see that the program

```
main()
{
    printf('hello, world\n');
}
```

indeed prints `hello, world`.

What about the program in Fig. 8.2 in the textbook?

It will print `hello, world`, for input n if and only if the equation

$$x^n + y^n = z^n$$

has a solution where x, y , and z are integers.

We now know that it will print `hello, world`, for input $n = 2$ (as $3^2 + 4^2 = 5^2$), and loop forever for inputs $n > 2$.

It took mathematicians 300 years to prove the latter, 'Fermat's last statement'. This statement was made by Fermat almost 400 years ago, but no proof was found until 1994 (Wiles).

A more general problem is 'does a given program, on a given input, print `hello, world?`'

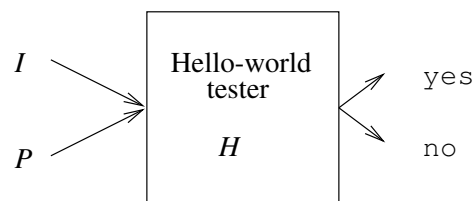
Any of the problems that mathematicians have not yet been able to resolve can be turned into a question of this form!

We will prove that no program H can tell whether or not a given program P with input I prints `hello, world` as its first output.

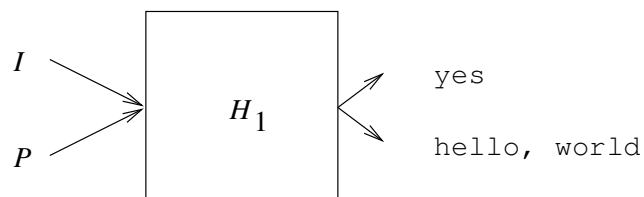
Can we then hope to have a program that proves the correctness of programs, i.e. is there a program that can tell whether or not a given program P with input I has output O ?

The hypothetical “hello, world” tester H

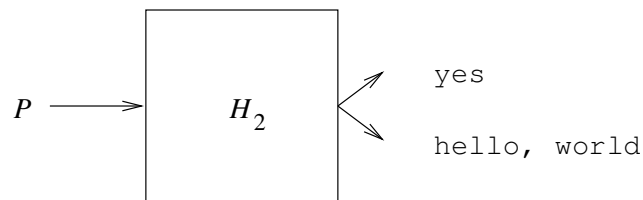
Suppose the following program H exists.



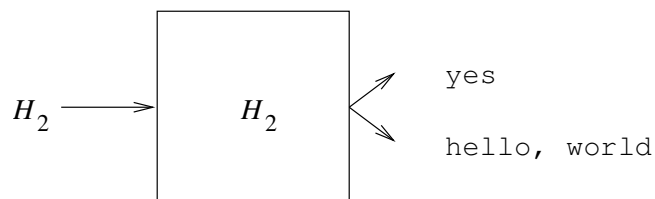
Modify the no print statement of H to hello, world. We get program H_1



Modify H_1 so that it uses its input P as both P and I . We get program H_2 .



Give H_2 as input to H_2 .



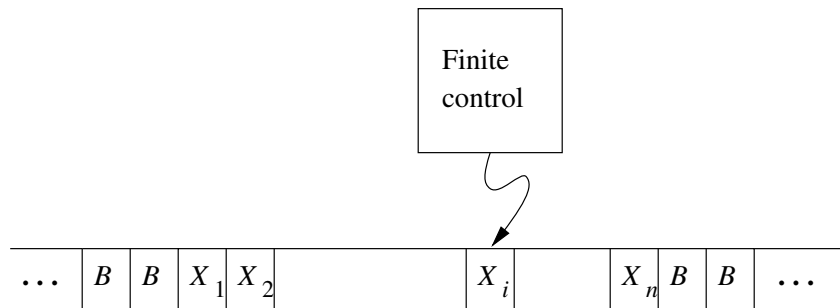
If H_2 prints yes, it should have printed hello, world as its first output.

If H_2 prints hello, world as its first output, it should have printed yes.

Thus H_2 cannot exist.

Consequently H cannot exist either.

The Turing Machine



A TM makes a *move* depending on its state, and the symbol under the tape head.

In a move, a TM will

1. change state
2. write a tape symbol in the cell scanned
3. move the tape head one cell left or right

A (deterministic) *Turing Machine* is a 7-tuple

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F),$$

where

- Q is a finite set of *states*,
- Σ is a finite set of *input symbols*,
- Γ is a finite set of *tape symbols*,
- δ is a *transition function* from $Q \times \Gamma$ to $Q \times \Gamma \times \{L, R\}$,
- q_0 is the *start state*,
- $B \in \Gamma$ is the *blank symbol*, and
- $F \subseteq Q$ is the set of final states.

Instantaneous Description

A *TM* changes *configuration* after each move.

We use Instantaneous Descriptions, ID's, to describe configurations.

An ID is a string of the form

$$X_1X_2\cdots X_{i-1}qX_iX_{i+1}\cdots X_n$$

where

1. q is the state of the TM.
2. $X_1X_2\cdots X_n$ is the non-blank portion of the tape
3. The tape head is scanning the i th symbol

The moves of a TM

We'll use $\vdash_M$ to indicate a move by M from a configuration to another.

- Suppose $\delta(q, X_i) = (p, Y, L)$. Then

$$\begin{array}{l} X_1 X_2 \cdots X_{i-1} q X_i X_{i+1} \cdots X_n \\ X_1 X_2 \cdots p X_{i-1} Y X_{i+1} \cdots X_n \end{array} \vdash_M$$

- If $\delta(q, X_i) = (p, Y, R)$, we have

$$\begin{array}{l} X_1 X_2 \cdots X_{i-1} q X_i X_{i+1} \cdots X_n \\ X_1 X_2 \cdots X_{i-1} Y p X_{i+1} \cdots X_n \end{array} \vdash_M$$

We denote the reflexive-transitive closure of $\vdash_M$ with $\vdash_M^*$.

The language of a TM

- A TM $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ *accepts* the language

$$L(M) = \{w \in \Sigma^* : q_0 w \vdash_M^* \alpha p \beta, p \in F, \alpha, \beta \in \Gamma^*\}$$

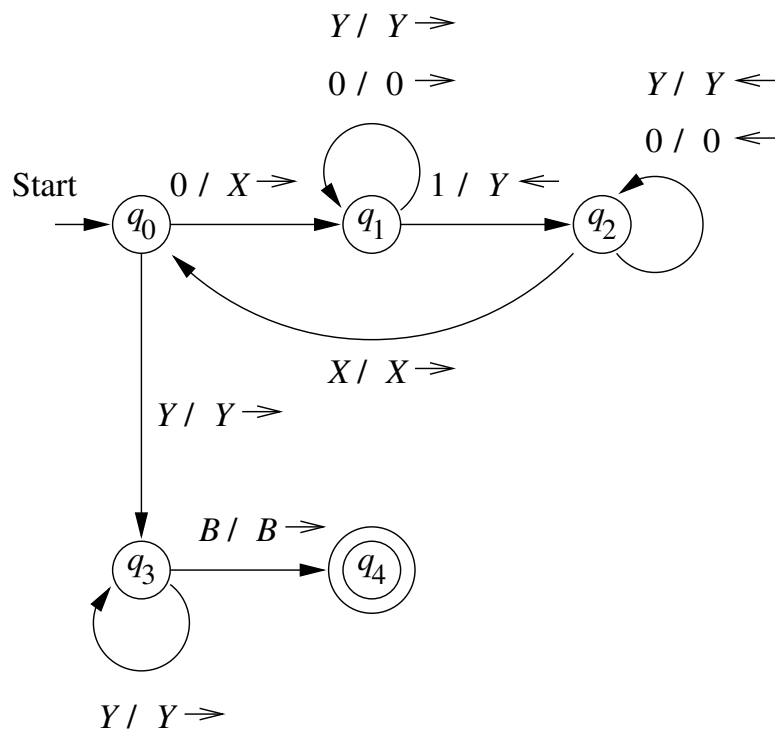
The set of languages accepted by Turing machines is often called the *recursively enumerable languages* or *RE languages*.

A TM for $\{0^n 1^n : n \geq 1\}$

$M = (\{q_0, q_1, q_2, q_3, q_4\}, \{0, 1\}, \{0, 1, X, Y, B\}, \delta, q_0, B, \{q_4\})$
 where δ is given by the following table

	0	1	X	Y	B
$\rightarrow q_0$	(q_1, X, R)			(q_3, Y, R)	
q_1	$(q_1, 0, R)$	(q_2, Y, L)		(q_1, Y, R)	
q_2	$(q_2, 0, L)$		(q_0, X, R)	(q_2, Y, L)	
q_3				(q_3, Y, R)	
$\star q_4$					(q_4, B, R)

We can represent M by the following *transition diagram*



A TM with “output”

The following TM computes

$$m - n = \max(m - n, 0)$$

	0	1	<i>B</i>
→ <i>q</i> ₀	(<i>q</i> ₁ , <i>B</i> , <i>R</i>)	(<i>q</i> ₅ , <i>B</i> , <i>R</i>)	
<i>q</i> ₁	(<i>q</i> ₁ , 0, <i>R</i>)	(<i>q</i> ₂ , 1, <i>R</i>)	
<i>q</i> ₂	(<i>q</i> ₁ , 1, <i>L</i>)	(<i>q</i> ₂ , 1, <i>R</i>)	(<i>q</i> ₄ , <i>B</i> , <i>L</i>)
<i>q</i> ₃	(<i>q</i> ₃ , 0, <i>L</i>)	(<i>q</i> ₃ , 1, <i>L</i>)	(<i>q</i> ₀ , <i>B</i> , <i>R</i>)
<i>q</i> ₄	(<i>q</i> ₄ , 0, <i>L</i>)	(<i>q</i> ₄ , <i>B</i> , <i>L</i>)	(<i>q</i> ₆ , 0, <i>R</i>)
<i>q</i> ₅	(<i>q</i> ₅ , <i>B</i> , <i>R</i>)	(<i>q</i> ₅ , <i>B</i> , <i>R</i>)	(<i>q</i> ₆ , <i>B</i> , <i>R</i>)
★ <i>q</i> ₆			

The transition diagram is

